בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Real Time Group

C for Embedded Systems

bennyc@rt-ed.co.il

Copyright @ 2018 Real Time College

דברי פתיחה

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

עם פתיחת מסלול הלימודים
צוות מכללת **Real Time** מאחל לכם חווית לימוד איכותית ופרודוקטיבית.

קורסי הכשרה מקצועית מכוונים לנתב את הקריירה שלך למסלול שבו תוכל/י להשתלב בתעשייה במקצוע שמעניין אותך ולמקסם את הידע בצורה אופטימלית האפשרית.

תרגישו נוח לפנות אלינו בכל נושא.

בני כהן
מנכ"ל

Copyright @ 2018 Real Time College

Index

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

1. Introduction 5
2. Variable And Constants 20
3. Expressions And Statements 42
4. Loops 60
5. Functions 71
6. Header Files 85
7. Arrays 89
8. Pointers 97
9. Strings 103
10. Dynamic Memory Allocations 116
11. Arguments to main 131
12. Pointer to Functions 134
13. Structures 137
14. Bitwise Operations 145
15. Big & Little Endian 151
16. Bit Fields & Unions 155
17. IO commands 166
18. the extern Keyword 179

Copyright @ 2018 Real Time College

Index

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

1. Static Variables 181
2. Linked Lists 185
3. Hash Tables 193
4. Sorting Methods 197

Copyright @ 2018 Real Time College

Introduction

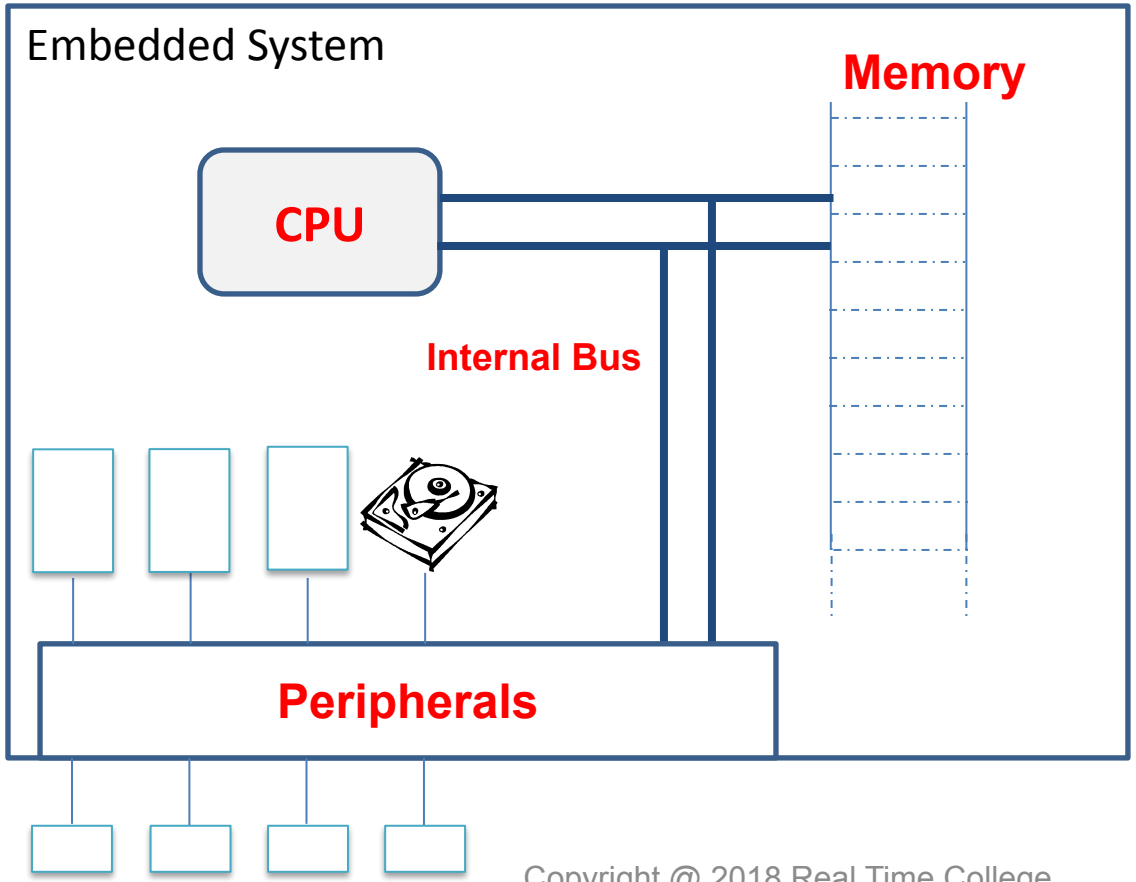
Copyright @ 2018 Real Time College

Introduction - Concepts of Embedded

(Hardware)

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Let us look at the concepts of Embedded programming through the eyes of a hardware developer. What do we need in order to make an embedded system work ?



1.CPU
2.Memory
3.Peripherals
4.Internal Bus
5.Power Supply
6.Clocking Unit
7.Storage Unit
8.….

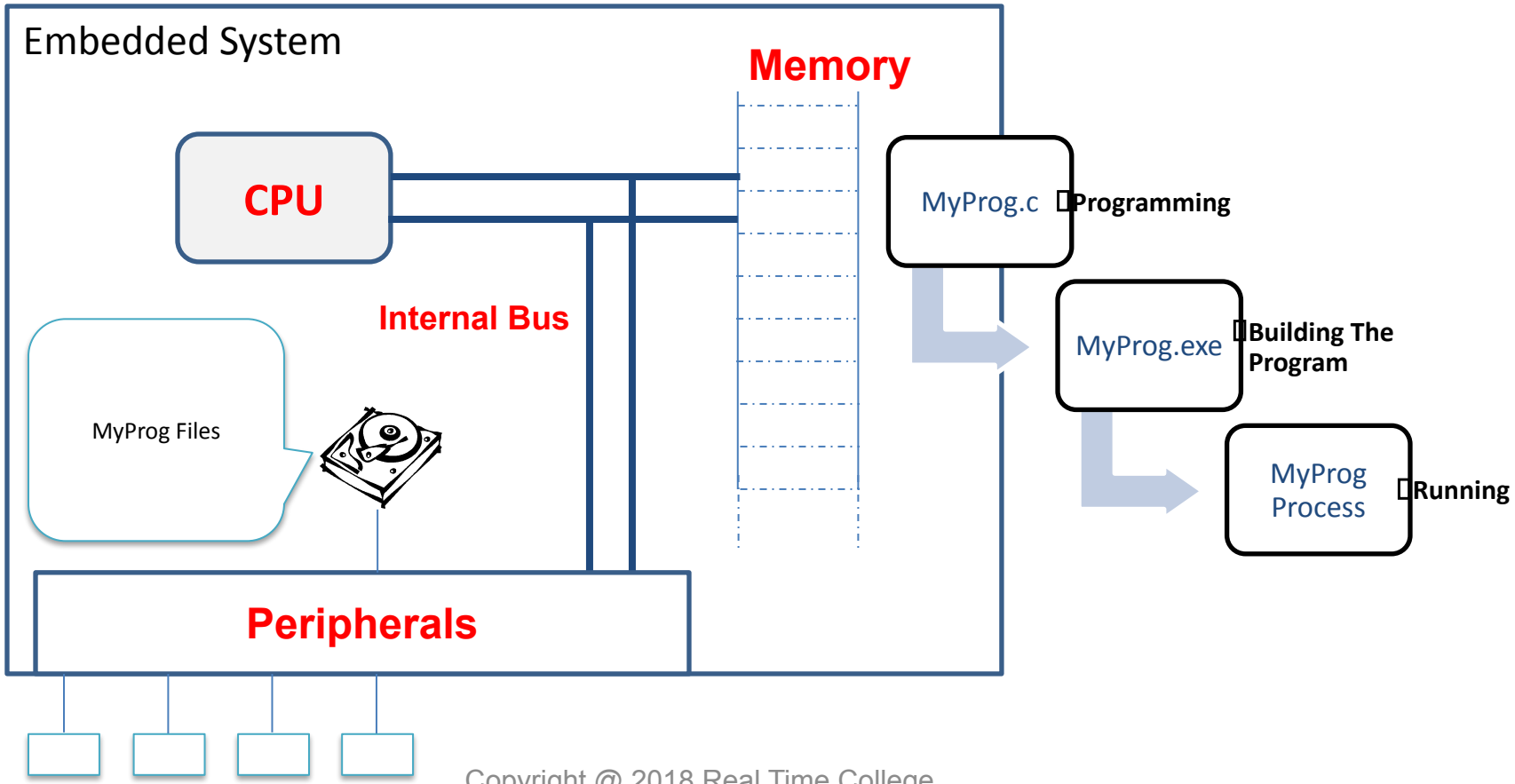
Copyright @ 2018 Real Time College

Introduction - Concepts of Embedded

(Programming)

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Lets say I want to run a program in my embedded system.
Better yet, I created one, and I want to run it … where is the program located ?



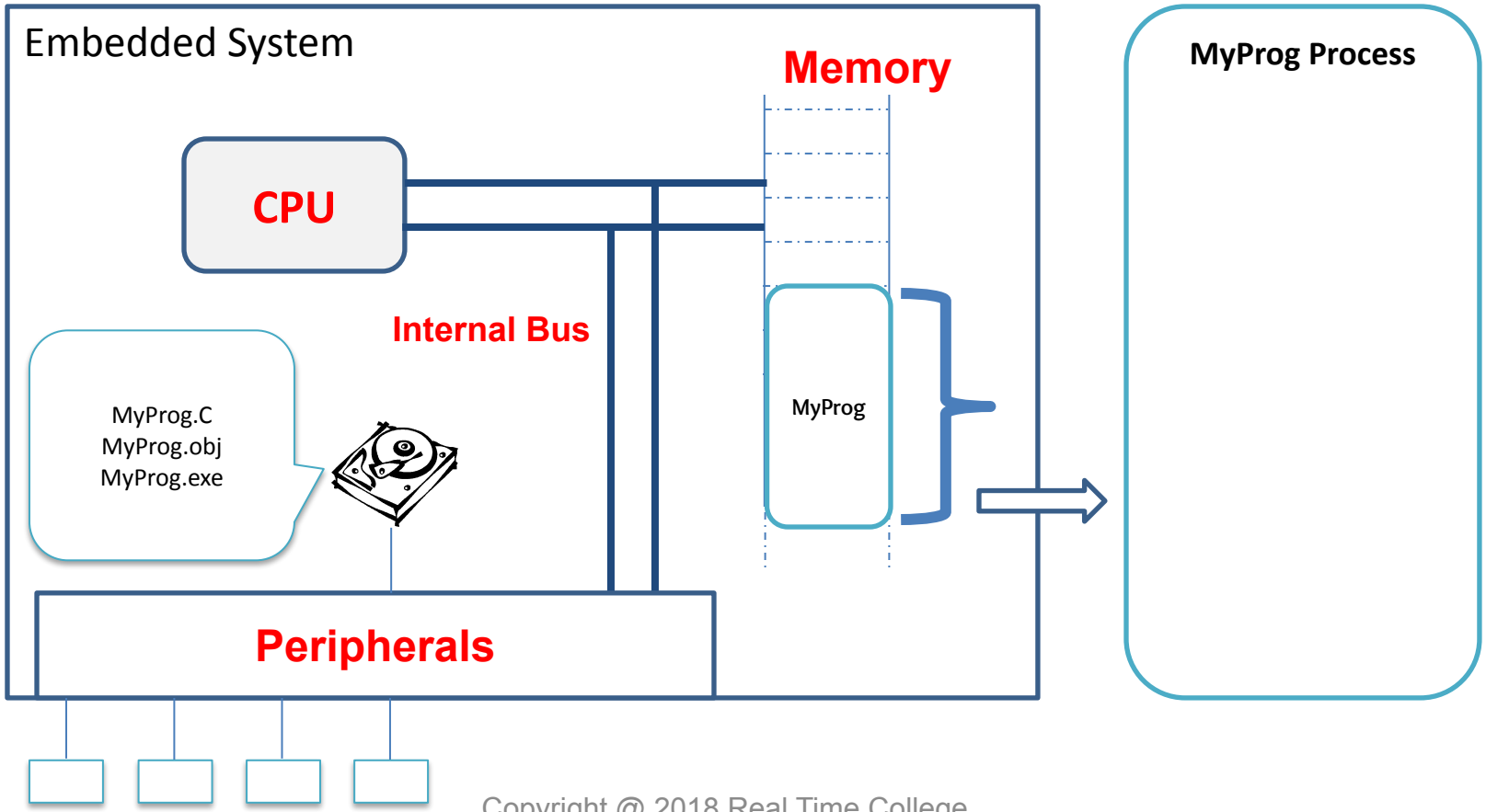
Copyright @ 2018 Real Time College

Introduction - Concepts of Embedded

(Program In Mem)

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Files will be located in the storage unit (HD, USB, Flash, CD/DVD etc..).
But a running program **must** run from memory ! Let's look closer at the process.



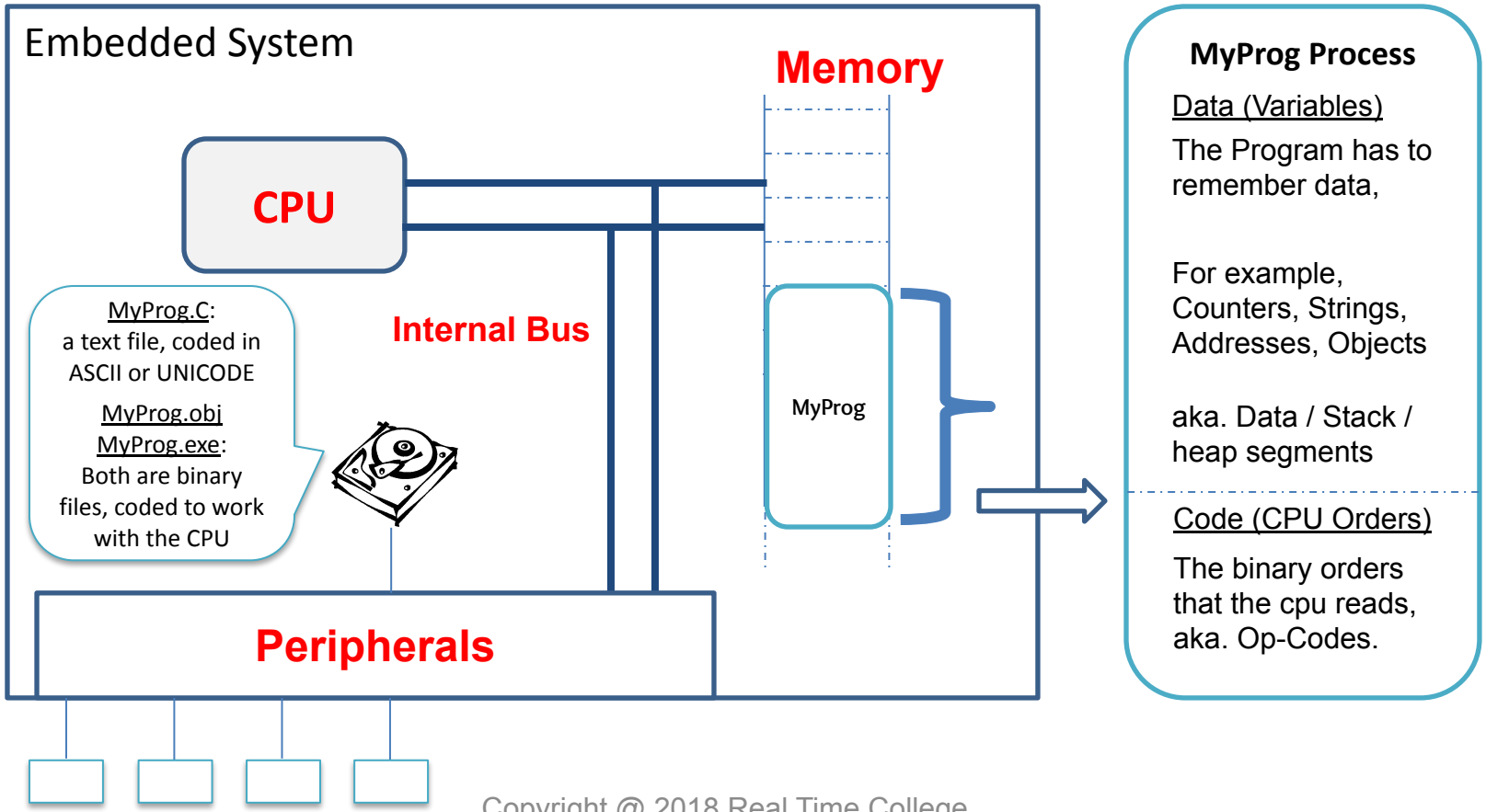
Copyright @ 2018 Real Time College

Introduction - Concepts of Embedded

(Program In Mem)

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Files will be located in the storage unit (HD, USB, Flash, CD/DVD etc..).
But a running program **must** run from memory ! Let's look closer at the process.



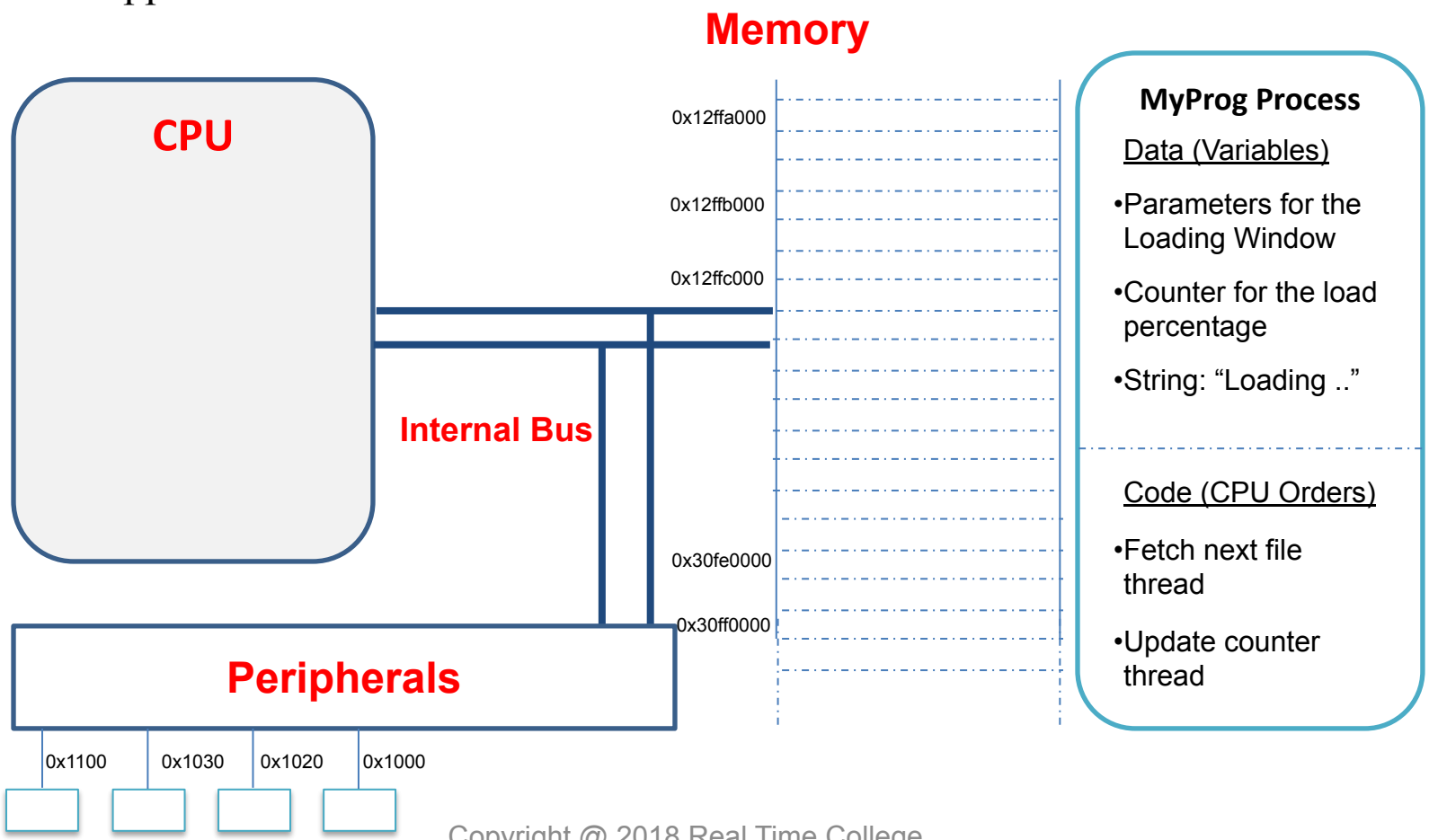
Copyright @ 2018 Real Time College

Introduction - Concepts of Embedded

(Program In Mem)

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Let's assume I have created an application. A simple example of “Loading ..” menu would appear like this:



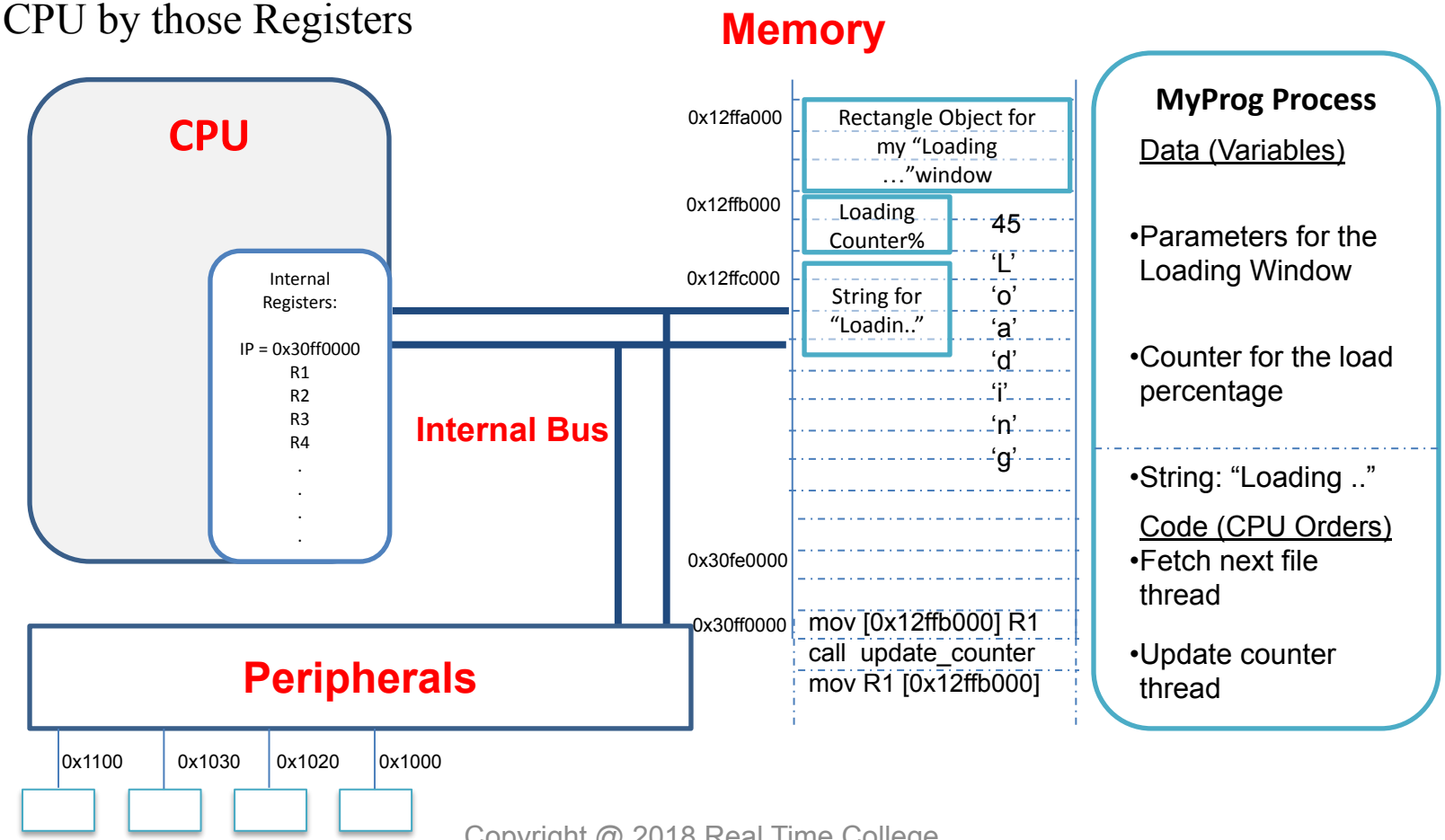
Copyright @ 2018 Real Time College

Introduction - Concepts of Embedded

(Program In Memory)

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

The CPU Knows everyone by it's address (memory and hardware). It also has special internal memory cells called CPU Registers. All the orders are executed within the CPU by those Registers



Copyright @ 2018 Real Time College

Hello World

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

What does the < > mean?

#include inserts another file. ".h" files are called "header" files. They contain stuff needed to interface to libraries and code in other ".c" files.

Can your program have more than one .c file?

```
#include <stdio.h>
/* The simplest C Program */
int main(int argc, char **argv)
{
  printf("Hello World\n");
  return 0;
}
```

This is a comment. The compiler ignores this.

The main() function is always where your program starts running.

Return '0' from this function

Prints out a message.
'\n' means "new line".

Copyright @ 2018 Real Time College

Hello World

The preprocessor runs before your compiler each time the compiler is invoked,
Looking for lines beginning with pound symbol (#).

main() - Every C/C++ program has a main() function.
A function is a block of code that performs one or more actions.

Each time your program starts, main() is called automatically.
main(), like all functions, must state what kind of value it gets and returns.

If the return value type is void – function will not return any value at all
int – a value is returned

NOTE: Standard **main** functions always return an **int** and not a **void**

Copyright @ 2018 Real Time College

the main() Function

main() is a Function. It's only special because it always gets called first when you run the program. You can pass Arguments to a function and it can return a Value.

Return type

Function Arguments

```
#include <stdio.h>
/* The simplest C Program */
int main(int argc, char **argv)
{
  printf("Hello World\n");
  return 0;
}
```

Calling a Function: "printf()" is just another function, like main().
It's defined for you in a "library", a collection of functions you can call from your program.

Returning a value

Copyright @ 2018 Real Time College

Basic I\O Commands

In order to communicate with the outside world we need I\O functions.

scanf() – gets characters from keyboard

Example: read from keyboard into a variable called var1 in decimal mode

Scanf("%d",&var1)

printf() – writes characters to monitor

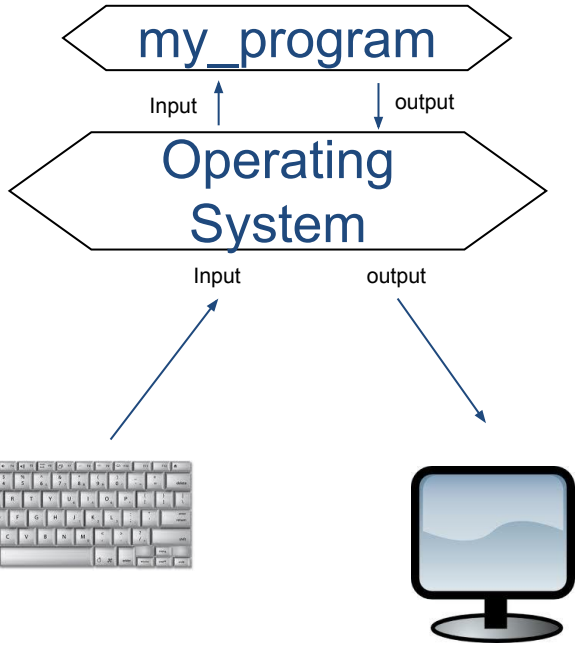
Example: write from a variable called var1 onto the screen in decimal mode

printf(" variable is equal %d", variable)

Copyright @ 2018 Real Time College

Basic I\O Commands (2)

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ



For example:

```
#include <stdio.h>

int main ()
{
   int decimal; // uninitialized value
   float pie=3.14;    // initialized value to pie

   printf("please enter a decimal number: ");
   scanf("%d", &decimal);

   printf("the number you entered is: %d\n", decimal);
   printf("but pie is closer to %f", pie);

   return 0;
}
```

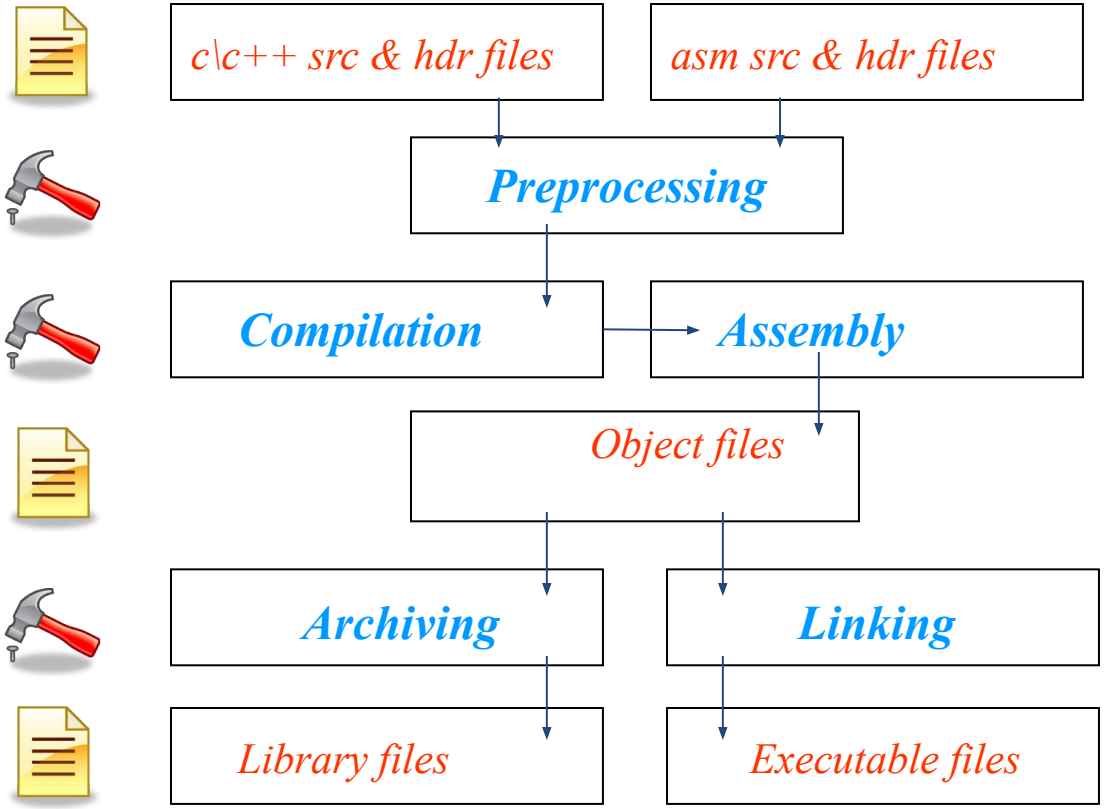
OUTPUT:
please enter a decimal number: 15
the number you entered is: 15
but pie is closer to 3.14

Copyright @ 2018 Real Time College

Compiling and Linking

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

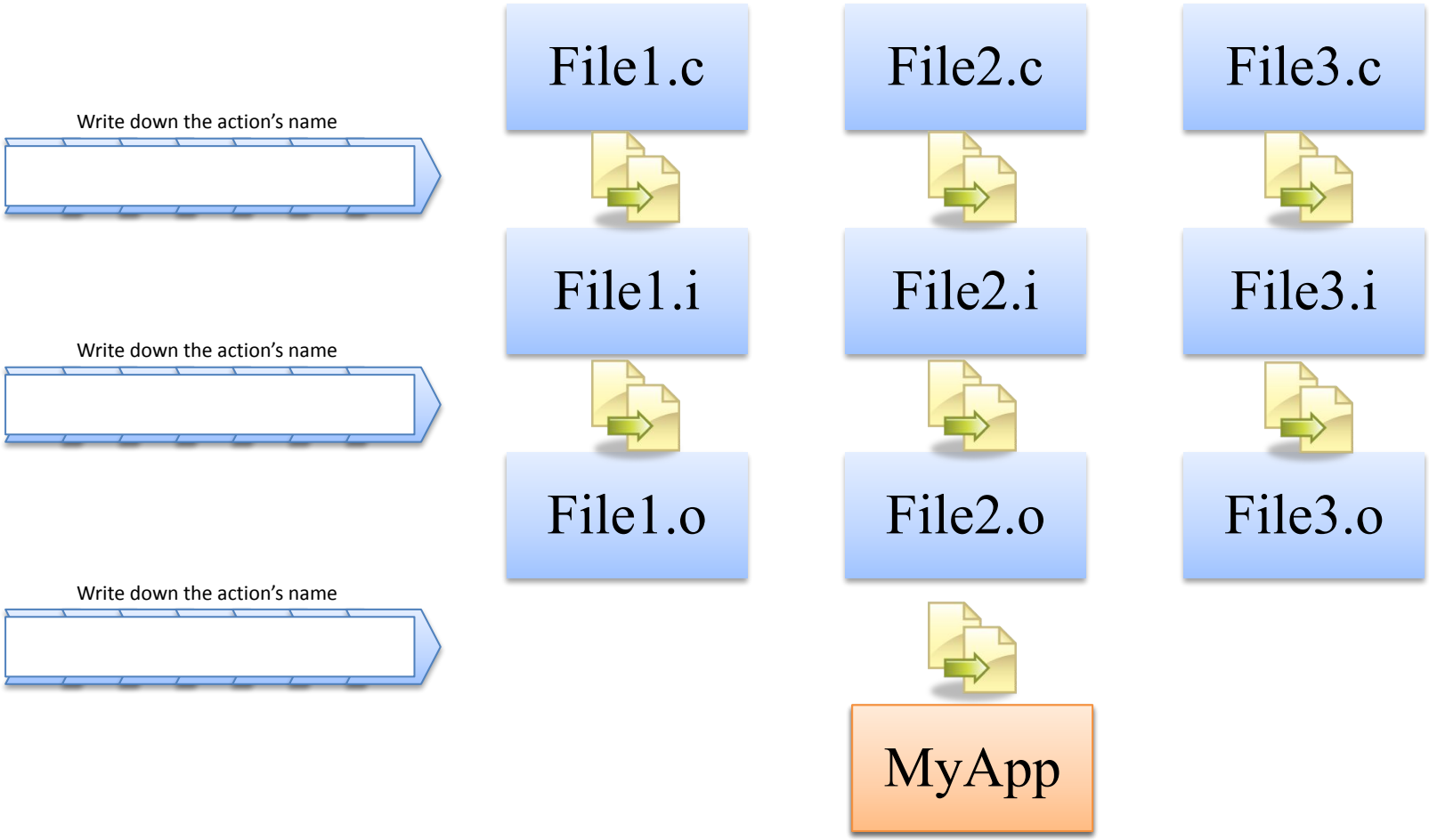
GNU cc (gcc) is the GNU project's compiler suite. It compiles programs written in C, C++. gcc gives the programmer extensive control over the compilation process. The compilation process includes up to four stages:



Copyright @ 2018 Real Time College

How does it work

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ



Copyright @ 2018 Real Time College

The Pre-Processor

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

```
#include <stdio.h>
/* The simplest C Program */
int main(int argc, char **argv)
{
  printf("Hello World\n");
  return 0;
}
```

Compilation occurs in two steps:
"Preprocessing" and "Compiling"

Preprocessing

```
typedef  unsigned long long int __dev_t;
typedef  unsigned int   __uid_t;
typedef  unsigned int   __gid_t;
typedef  unsigned long int   __ino_t;
```

Compile

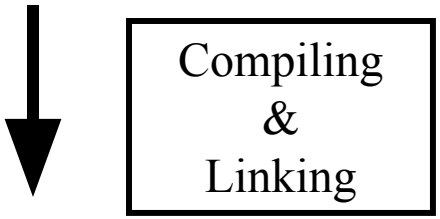
In Preprocessing, source code is "expanded" into a larger form that is simpler for the compiler to understand. Any line that starts with '#' is a line that is interpreted by the Preprocessor.

- **Include files are "pasted in" (#include)**
- **Macros are "expanded" (#define)**
- **Comments are stripped out (/* */ , //)**
- **Continued lines are joined (\ )**

Copyright @ 2018 Real Time College

the Compiler


```
typedef  unsigned long long int __dev_t;
typedef  unsigned int   __uid_t;
typedef  unsigned int   __gid_t;
typedef  unsigned long int   __ino_t;
```



An executable file

The compiler then converts the resulting source code into binary opcode the CPU can run directly

The binary file created is really series of Opcodes for the CPU to run

The Compiler has to be Architecture specific.

Copyright @ 2018 Real Time College

Variables And Constants

Copyright @ 2018 Real Time College

Variables (1)

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

A variable is a place to store information,
its located in your computer's memory (you can read or write to it).

The computer's memory can be viewed as a series of slots (one Byte each), Every Byte has an address.

Declaring a Variable - reserves one or more Bytes.
The variable's name (`myVar` in this case) is a label on one of these cubbyholes, so that you access it easily without knowing its actual memory address.

`myVar`

For example, `myVar` starts at memory address 0xFF08 and its size can take up one or more memory addresses.

Addr	Value
0xFF00	
0xFF01	
0xFF02	
0xFF03	
0xFF04	(72)
0xFF05	(101)
0xFF06	'l' (108)
0xFF07	'l' (108)
0xFF08	(111)
0xFF09	'\n' (10)
0xFF0A	'\0' (0)

Copyright @ 2018 Real Time College

Variables (2)

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

A Variable names a place in memory where you store a Value of a certain Type.
You define a variable by giving it a name and specifying the type, and optionally an initial value (example below).
Multiple definition can be done by using a comma seperator (,).

```
char x;
char y='e', y1, y2;
```

Initial value of x is undefined

Initial value

Name

What names are legal?

Type (for example: char. occupies a single byte)

Additional attributes can be set. Keep on reading ….

The compiler puts them somewhere in memory.

Addr	Value
0xFF00	
0xFF01	
0xFF02	
0xFF03	
0xFF04	'e' (101)
0xFF05	(72)
0xFF06	'l' (108)
0xFF07	'l' (108)
0xFF08	(111)
0xFF09	'\n' (10)
0xFF0A	'\0' (0)

Copyright @ 2018 Real Time College

Variables (3)

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

When defining a new variable, specifying the Variable Type (char \ int \ float etc..) is mandatory! this tells the compiler the amount of memory to allocate for this variable from RAM.

we can also tell the compiler the variables Initial Value (optional).

```
char myVariable = 111;
```

Addr	Value
0xFF00	
0xFF01	
0xFF02	
0xFF03	
0xFF04	
0xFF05	(111)
0xFF06	
0xFF07	

`myVariable`

NOTE: RAM is Random Access Memory.
When a program runs, it is loaded into RAM from the disk,
All variables are also created in RAM.
(Variables are not created in FLASH)

Lst0302.c

What more attributes are there for variables ?

Copyright @ 2018 Real Time College

Variable names

The variable name can be any combination of letters, numbers, and also the char '_', but cannot contain white spaces. The first letter of a variable cannot be numeric.
The name should tell you what the variable is used for; using good names makes it easier to understand the flow of your program,
You cannot use Keywords as variables (if, while…)
It is a common convention to use a lower case first character name for a regular variable. Each company / community has its own standards of coding style.
e.g: int **m**yAge, **h**isAge, **a**geing;

NOTE: C is case-sensitive -
uppercase and lowercase letters are considered to be different.

A variable named age is different from Age, which is different from AGE.

Copyright @ 2018 Real Time College

Variable Types

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Variable types can be divided into

- **integer**
- **floating-point**
- **character**

the table shows the variable type, how much room it takes in memory.

The values that can be stored are determined by the size of the variable types.

Type	*Size in bytes*
`int` Signed / unsigned) (doesn't change size	bytes 4 (arch dependent)
`short`	bytes 2
`long`	bytes 4
`long long`	bytes 8
`float`	bytes 4
`double`	bytes 8
`long double`	bytes 16
`char`	bytes 1

NOTE: Variable sizes might change between architectures.

Copyright @ 2018 Real Time College

sizeof()

This keyword function is provided by your compiler, and it tells you the size of the object you pass in as a parameter (in bytes).
Example:

```
char a3;
short a2;
int a1;
1 == sizeof(a3) == sizeof(char )
2 == sizeof(a2) == sizeof(short )
4 == sizeof(a1) == sizeof(int)
```

Lst0201.cxx

Copyright @ 2018 Real Time College

Coding Style


```
main ()
{
<tab>   int myWidth;
<tab>   int myLength;
..      int itsArea;

        itsArea=myWidth*myLength;
}
```

```
main () {
<tab>   int my_width;
<tab>   int my_length;
..      int its_area;

        its_area=my_width*my_length;
}
```

NOTE :
There are two popular conventions: `myCar` or `my_car`.
The first latter form is called camel-notation, because the capitalization looks something like a camel's hump.
Some people find the underscore character (`my_car`) to be easier to read.

Copyright @ 2018 Real Time College

Signed / Unsigned (1)

integer types come in two varieties:

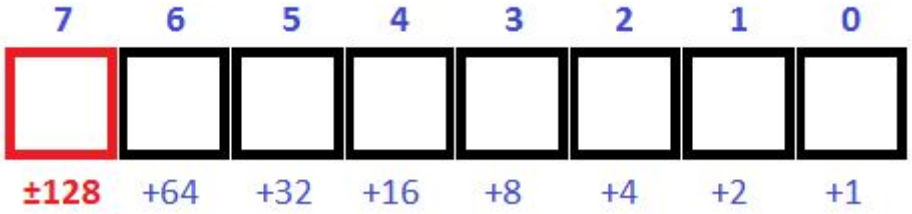
signed / unsigned

Integers (`short` and `long`) without the word "unsigned" are assumed to be `signed`.

`Signed` integers - are either negative or positive: (short e.g. -32,768 to 32,767).

`Unsigned` integers - are always positive: (short e.g. 0 to 65,535)

you have the same number of bytes for both `signed` and `unsigned` integers, so the largest number you can store in an `unsigned` integer is twice as big as the largest positive number you can store in a `signed` integer.



Copyright @ 2018 Real Time College

Signed / Unsigned (2)


```
signed char ch = 130;            // same as ch = -126;
```

7	6	5	4	3	2	1	0
1	0	0	0	0	0	1	0
±128	+64	+32	+16	+8	+4	+2	+1

```
unsigned char ch = 130;          // same as ch = -126
```

7	6	5	4	3	2	1	0
1	0	0	0	0	0	1	0
±128	+64	+32	+16	+8	+4	+2	+1

If not specified, the "signedness" of plain char will be architecture dependant. (either signed or unsigned)
For example, On GCC compiler On X86 architecture it's usually signed.

Copyright @ 2018 Real Time College

Signed / Unsigned (3)

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Signed

- Sign bit on makes a value negative
- Positive signed = Unsigned values
- Adding X and –X in binary results in 0

For example:
3 - 3 = 0

0000 0011 equals 3
1111 1101 equals -3

0000 0000

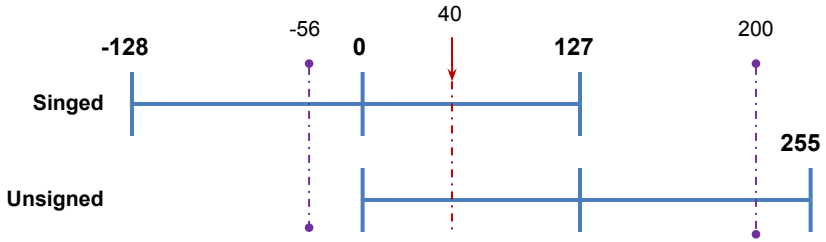
Unsigned

- Sign bit on makes no difference
- Cannot save a negative number
- The highest number will be -1

For example:
unsigned char ch = -1;

ch ← 1111 1111
Which is 255

Try it in short/long int



Copyright @ 2018 Real Time College

Short \ Long

If there is any chance that the value you'll want to put into your variable will be too big for its type, use a larger type.

When an `unsigned` integer reaches its maximum value, it wraps around and starts over, much as a car's speedometer might.

<u>Example:</u>

```
unsigned short int smallNumber;
smallNumber = 65535;//0xffff
smallNumber++
```

Lst0204.cxx and
Lst0205.cxx

Copyright @ 2018 Real Time College

typedef

It can become tedious, repetitious, and, most important, error-prone to keep writing `unsigned short int`.
C/C++ enables you to create an alias for this phrase by using the keyword **`typedef`**, which stands for type definition.
`typedef` is used by writing the keyword `typedef`, followed by the existing type and then the new name.
It is considered a better practice to distinguish types from variables by naming them with a capital letter as a first letter, or add _t at the end of a new type. Also common types are: U8 / U16 / U32 or uint / ushort / ulong or uint8 / uint16 / uint32 etc…

<u>Example</u>

```
typedef unsigned short int U16; //typedef defined
U16 width = 5;
```

Lst0203.c

Copyright @ 2018 Real Time College

Characters and Numbers

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

When you put a character, for example, `'a'`, into a `char` variable, what is really there is just a number between 0 and 255. The compiler knows, how to translate back and forth between characters.

- The value/letter relationship is mapped according to a known char-map called **ASCII table**. for example lowercase "a" is assigned the value `97` decimal.
- **`Important`** - there is a big difference between the value `5` and the character `'5'`. The letter is actually valued at `53`, much as the letter `'a'` is valued at `97`.
- The compiler recognizes some special characters for formatting. You put these into your code by typing the backslash (called the escape character), followed by the character.

The Escape Characters.

`\n`	new line
`\t`	tab
`\b`	backspace
`\"`	double quote
`\'`	single quote
`\?`	question mark
`\\`	backslash

Lst0206.cxx

See the full map at Lesson1 ascifull

Copyright @ 2018 Real Time College

Casting

C/C++ can help you convert between different types, this is called casting.
There are two forms of casting. Implicit and Explicit.
Implicit: the type is converted according to the receiver's variable type.

```
// Implicit example:
int a = 5.25;     // a equals 5 after init.
```

Explicit: the type is specifically converted to a different type.

```
// Explicit example:
float f = (float) 5.25;     // f equals 5.25 after init.
/* floating point literal default type (like 5.25) is double.  */
```

Syntax: (casting type) <expression>

Copyright @ 2018 Real Time College

I/O Commands printf/scanf formats

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

printf and scanf uses the following format for it's printable text:
%[*parameter*][*flags*][*width*][.*precision*][*length*]*type*
Most commonly used are: %c, %d, %f, %s, %x, %p

Conversion Character	Displays Argument (Variable's Contents) As
%c	Single character *(char)*
%d	Signed decimal integer *(int)*
%e , %E	Signed floating-point value in E notation (example: 10000 -> 1e04)
%f , %F	Signed floating-point value *(float)*
%g , %G	Signed value in %e or %f format, whichever is shorter
%i (same as %d)	Signed decimal integer *(int)*
%o	Unsigned octal (base 8) integer *(int)*
%s	String of text *(char *)*
%u	Unsigned decimal integer *(int)*
%x , %X , %p	Unsigned hexadecimal *value* / *(type *)*, %p used for pointers.

%[Capital letters] prints the hex characters in Upper-Case

Copyright @ 2018 Real Time College

I/O Commands printf/scanf formats

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

printf is able to print text and variables, or just one of the two. As for scanf though, it is only able to scan for variables.

Using width will cause the print function to allocate in advance space for the printed variable.

Using precision will cause the print function to show X characters of floating points.

%[*parameter*][*flags*][*width*][.*precision*][*length*]*type*

For example:

```
printf("num1: %3d\n",1);
printf("num2: %3d\n",12);
printf("num3: %3d\n",123);
printf("10/3 = %f, pretty accurate\n", 10/3.);
printf("10/3 = %6.2f , easier to read\n", 10/3.);
```

OUTPUT

```
num1:   1
num2:  12
num3: 123
10/3 = 3.33333333, pretty accurate
10/3 =   3.33, easier to read
```


Copyright @ 2018 Real Time College

Table of printf format specifiers:

%[parameter][flags][width][.precision][length]type

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

%hhu	unsigned char	%hhd	char
%hu	unsigned short int	%hd	short int
%u	unsigned int	%d	int
%lu	unsigned long int	%ld	long int
%llu	unsigned long long int	%lld	long long int
%e	float (scientific)	%f	float (normal display)
%le	double (scientific)	%lf	double (normal display)
%Le	long double (scientific)	%Lf	long double (normal display)

Copyright @ 2018 Real Time College

Constants

Like variables, constants are data storage locations. Unlike variables, constants don't change.
You must initialize a constant when you create it, and you cannot assign a new value to it later. There are two ways to declare constants in C/C++. The old, traditional, and obsolete way is with a preprocessor directive, `#define`.

```
#define studentsPerClass 15
```

Every time the preprocessor sees the word `studentsPerClass`, it puts in the text `15`. This type of constant is not a variable, and does not have space in memory. It is simply an expression of a macro.
there is a new, much better way to define constants in C/C++:

```
const int studentsPerClass = 15;
```

This method has several advantages in making your code easier to maintain and in preventing bugs. The biggest difference is that this constant has a type, and the compiler can enforce that it is used according to its type.

Copyright @ 2018 Real Time College

Enumerated Constants

Enables to create new types and to define variables of those types whose values are restricted to a set of possible values.

For example:

declare COLOR to be an enumeration, and that there are five values for COLOR: RED, BLUE, GREEN, WHITE, and BLACK.

```
enum COLOR { RED, BLUE, GREEN, WHITE, BLACK };
```

1. Makes COLOR the name of an enumeration, that is, a new type.

2. Makes RED a symbolic constant with the value 0, BLUE a symbolic constant with the value 1, GREEN equals 2, and so forth.

It is possible to initialized the constants with a particular value, however, and those that are not initialized will count upward from the ones before them.

```
enum Color { RED=100, BLUE, GREEN=500, WHITE,BLACK=700 };
```

RED will have the value 100; BLUE =101;

GREEN= 500; WHITE = 501; and BLACK=700.

Lst0207.c and Lst0208.c

Copyright @ 2018 Real Time College

Enumerated Constants (2)

After creating the type, using it (aka. Defining a variable in the enum's type) will have either one of the following syntax:

1.

```
enum COLOR { RED, BLUE, GREEN, WHITE, BLACK };
enum COLOR screen_background = BLUE;
```

2.

```
enum COLOR { RED, BLUE, GREEN, WHITE, BLACK }
    clr_background = RED, clr_text=GREEN, clr_window;
clr_window = WHITE;
```

<u>This however can be shorten using typedef:</u>

```
typedef enum Color { RED, BLUE, GREEN, WHITE,BLACK } Color_t;
Color_t clr_bground = RED, clr_text=GREEN, clr_window;
clr_window = WHITE;
```

Lst0207.c and Lst0208.c

Copyright @ 2018 Real Time College

Creating a structured object

This subject will be deeply covered later on, but just as an intro, you can group several variables into one structured object using the keyword struct. The syntax is exactly as enum. Again, for more information skip ahead for the structs chapter.

1.
```
struct Point { int x, y; };
//or: struct Point { int x; int y; };
struct Point my_point;
my_point.x = 5;
```
2.
```
struct Point { int x, int y } my_point;
my_point.y = 5;
```

<u>Shorten using typedef:</u>

```
typedef struct Point { int x, int y } Point_t;
Point_t my_point;
my_point.x = 5;
```


Copyright @ 2018 Real Time College

Expressions And Statements

Copyright @ 2018 Real Time College

Expressions

Anything that evaluates to a value is an expression in C.
An expression is said to return a value.
For example
3+2 returns the value 5 and so it is an expression.

```
x = a + b;
```

This example is evaluated in the following order:
1.Add a to b.
2. Assign the result of the expression a + b to x.

```
y = x;
```

3.Assign the result of the assignment expression x (= a + b) to y.

4.1 - expressions

Copyright @ 2018 Real Time College

Operators (1)

An operator is a symbol that causes the compiler to take an action.
Operators act on operands. The operators listed by their categories as follows:

Arithmetic operators	+ - * / % ++ --
Comparison & relational operators	== != <= >= < >
Logical operators	&& || !
Bitwise operators	& | ~ ^ << >>
Assignment operators	= [Arithmetic/Bitwise]=
Member and pointer operators	[] & * -> .
Special operators	?: sizeof() (type)casting

Copyright @ 2018 Real Time College

Operators (2)

As we mentioned, there are several different categories of operators. Two of these categories are

- **Assignment operators.**
- **Arithmetic operators**.

The assignment operator (=) causes the operand on the left side of the assignment operator to have its value changed to the value on the right side of the assignment operator. Constants are r-values. They cannot be l-values. you can write

x = 35; // ok

but you can't legally write

35 = x; // error, not an lvalue!

Copyright @ 2018 Real Time College

Operators (3)

There are five mathematical operators: addition (+),
subtraction (-),
multiplication (*),
division (/),
modulus (%).

Shortcuts:

x += y	**assign (x+y)**	**to x => x=x+y**
x -= y	**assign (x-y)**	**to x=> x=x-y**
x *= y	**assign (x*y)**	**to x => x=x*y**
x /= y	**assign (x/y)**	**to x => x=x/y**
x %= y	**assign (x%y)**	**to x => x=x%y**

**** 4.2. subtraction and overflow.**

Copyright @ 2018 Real Time College

Integer Division and Modulus

Integer division is different from everyday division.
When you divide 21 by 4, the result is a real number (a number with a fraction).
Integers don't have fractions, and so the "remainder" is lopped off. The answer is therefore 5. To get the remainder, you take 21 modulus 4 (21 % 4) and the result is 1.
The modulus operator tells you the remainder after an integer division.

Beware of division:

- If second argument is integer, the result will be integer (rounded):
  5 / 10 → 0 *whereas* 5 / 10.0 → 0.5
- Division by 0 will cause a FPE

Copyright @ 2018 Real Time College

Increment and Decrement

In C, increasing a value by 1 is called incrementing,
and decreasing by 1 is called decrementing.
There are special operators to perform these actions.
The increment operator (++) increases the value of the variable by 1,
and the decrement operator (--) decreases it by 1

```
var++;  var = var + 1;      var += 1; //all the same
Var--;  var = var - 1;      var -= 1;
```

The rules of precedence are clearly defined but
often difficult to remember or non-intuitive.
When in doubt, add parentheses to make it explicit.

Copyright @ 2018 Real Time College

Prefix and Postfix

Both the increment operator (++) and the decrement operator(--) come in two varieties: prefix and postfix.

Prefix: you have told the compiler to increment x (making it 5+1= 6) and then fetch that value and assign it to a.(x=x+1 and then a=x)

int a = ++x;

Postfix: fetch the value of x and assign it to a, then inc x .

(a=x and then x=x+1)

Note the difference between ++x and x++:

```
int x=5;
int y;
y = ++x;
/* x == 6, y == 6 */
```

```
int x=5;
int y;
y = x++;
/* x == 6, y == 5 */
```

****4.4. prefix and postfix.**

Copyright @ 2018 Real Time College

Comparison Operators

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

In C, zero is considered false, and all other values are considered true, although true is usually represented by 1. If an expression equal to zero, it is false. If it is non zero, it is true.

The relational operators are used to determine whether two numbers are equal, or if one is greater or less than the other. Every relational statement evaluates to either:
1 (TRUE) or 0 (FALSE).

Example: int x=4;
(x < 5) → (4 < 5) → 1 <true>
(x < 4) → (4 < 4) → 0 <false>
((x < 5) || (x < 4)) → (<true> || (x < 4)) → <true>

Equals	==
Not Equals	!=
Greater	>
Greater or Equal	>=
Less	<
Less or Equal	<=

Logical OR and AND operators are represented by:
|| = OR, && = AND. It will be discussed shortly.

Copyright @ 2018 Real Time College

The if Statement

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

The `if` statement enables you to test for a condition (such as whether two variables are equal) and branch to different parts of your code, depending on the result.

```
if (expression)
    statement;

if (bigNumber > smallNumber)
{
statement1;
    statement2;
    statement3;
}
```

```
== equal to
<  less than
<= less than or equal
>  greater than
>= greater than or equal
!= not equal
&& logical and
|| logical or
!  logical not
```

****4.5. branching based on relational operators.**

Copyright @ 2018 Real Time College

The else keyword

Often your program will want to take one branch if your condition is true, another if it is false.

```
if (expressio) {
    statements;
}
else
{
    statements;
}
```

WARNING: Many C/C++ programmers confuse the assignment operator (=) with the equals operator (==). This can create a nasty bug in your program.

****4.6 the else keyword.**

Copyright @ 2018 Real Time College

Advanced if Statements


```
if (expression1) {
    if (expression2){
      statement1;
    }
    else
    {
        statement2;
    }
}
else
{
    statements;
}
```

**4.6. A complex, nested if statement.
**4.7. how braces help clarify which else statement goes with which if statement.
**4.8. the proper use of braces with an if statement

Copyright @ 2018 Real Time College

else if Statements

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

```
if (expression1)                    {

    statement1;
}
else if (expression2)
{                   statement2;
}
else if (expression3)
...
else
{
    statement;
}
```


Copyright @ 2018 Real Time College

Logical Operators

A program might need to determine that couple of conditions are true, or that some other condition is true, in order to take an action.
Imagine a sophisticated alarm system that has this logic: "If the door alarm sounds AND it is after six p.m. AND it is NOT a holiday, OR if it is a weekend, then call the police."
C/C++'s three logical operators are used to make this kind of evaluation.

Conditions usage ..

```
&& -  logical and
|| -  logical or
!  -  logical not
```

Bitwise operators:

```
&  - bitwise and
|  - bitwise or
^  - bitwise xor
~  - bitwise not
<< - shift left
>> - shift right
```


Copyright @ 2018 Real Time College

Logical AND

A logical `AND` statement evaluates two expressions, and if both expressions are true, the logical `AND` statement is true .
If it is true that you are hungry, AND it is true that you have money, THEN it is true that you can buy lunch. Thus,

```
if ( (x == 5) && (y == 5) )
```

would evaluate `TRUE` if both `x` and `y` are equal to 5, and it would evaluate `FALSE` if either one is not equal to 5.
Note that both sides must be true for the entire expression to be true.
Note that the logical `AND` is two `&&` symbols, A single `&` symbol is a different operator.

Don't confuse & and &&..
1 & 2 → 0 *whereas* 1 && 2 → <true>

Copyright @ 2018 Real Time College

Logical OR

A logical `OR` statement evaluates two expressions. If either one is true, the expression is true.

If you have money OR you have a credit card, you can pay the bill. You don't need both money and a credit card; you need only one, although having both would be fine as well.

Thus,

```
if ( (x == 5) || (y == 5) )
```

evaluates `TRUE` if either `x` or `y` is equal to 5, or if both are.

Note that the logical `OR` is two `||` symbols. A single `|` symbol is a different operator,

Don't confuse | and ||..
1 | 2 → 3 *whereas* 1 || 2 → <true>

Copyright @ 2018 Real Time College

Logical NOT

A logical `NOT` statement evaluates `true` if the expression being tested is false. Again, if the expression being tested is false, the value of the test is `TRUE`! Thus

```
if ( !(x == 5) )
```

is true only if `x` is not equal to 5. This is exactly the same as writing

```
if (x != 5)
```

Don't confuse = and == The compiler might not warn.

NOTE: It is often a good idea to use extra parentheses to clarify what you want to group. Remember, the goal is to write programs that work and that are easy to read and understand.

Copyright @ 2018 Real Time College

Ternary operator (if statement shortcut)

The `if` statement could be shortened to an if expression using the ternary operator – '?'. Consider this as an if expression.

```
if (expression) // condition
    statement; // if true
else
    statement; // if false
```

Syntax:

(expression-condition)?(expression-if-true):(expression-if-false)

Example:

char a='a', A='A', bigger=0; // a = 'a' (97), A = 'A' (65), bigger = 0

bigger = (a>A)?(a):(A); // bigger ← 'a' (97)

Copyright @ 2018 Real Time College

The switch Statement

you know how to write *`if`* and *`if/else`* statements. These can become quite confusing when nested too deeply, C offers an alternative. Unlike `if`, which evaluates one value, *`switch`* statements allow you to branch on any of a number of different values.

The general form of the `switch` statement is:

```
switch (expression)
{
    case valueOne: statement;
    break;
    case valueTwo: statement;
    break;
    ....
    case valueN: statement;
    break;
    default: statement;
}
```

**** 4.9. the switch statement.**

****4.9. switch case in a forever loop.**

Copyright @ 2018 Real Time College

Loops

Copyright @ 2018 Real Time College

Looping

Some programs need to repeatedly act on the same data. This is called iteration. Iteration means doing the same thing again and again. The principal method of iteration is the loop.

The type of loops used are:

- While \ Do - While loops
- For loops
- Goto - Warning: Do not use!

- As a rule of thumb **goto** should never be used no matter how tempting or easy it appears. Using **goto** can eventually lead to a buggy code that's hard to read & maintain!

- There are some fringe rare cases **goto** has debatable legitimate uses, but you should always look for alternative solutions!

Copyright @ 2018 Real Time College

while Loops

A `while` loop causes your program to repeat a sequence of statements as long as the starting condition remains true.
The syntax for the `while` statement is as follows:

```
while ( condition )
{
 statement;
 statement;
}
```

When condition evaluates to `TRUE` (`1`), statement is executed, and then condition is tested again. This continues until condition tests `FALSE`, at which time the `while` loop terminates and execution continues on the first line below statement.

****6.2. while loops.**

Copyright @ 2018 Real Time College

Complicated while Statements

The condition tested by a `while` loop can be as complex as any legal C/C++ expression. This can include expressions produced using the logical `&&` (AND), `||` (OR), and `!` (NOT) operators.

```
while ( (i<MAX && strlen(s)<=10) || !(kbhit == 1 && key==ESC ) )
{
 .....
}
```

****6.3. Complex while loops.**

Copyright @ 2018 Real Time College

The break Statement

break; causes the immediate end of a `while` or `for` loop.
Execution jumps to the closing brace.
Example

```
while (condition)
{
    if (condition2)
    {
      statements;
      break;
    }
   statements; // will not be executed!!!
}
[statements];
```

**6.4. break and continue.

Copyright @ 2018 Real Time College

The continue Statement

`continue;` causes end of a single iteration in `while` or `for` loop.
Execution jumps to the condition of the loop.
Example

```
while (condition)
{
    if (condition2)
    {
      statements;
      continue;
    }
   statements; // will not be executed!!!
}
```

**6.4. break and continue.

Copyright @ 2018 Real Time College

The do...while Statement

The do...while loop executes the body of the loop before its condition is tested and ensures that the body always executes at least one time.
The syntax for the do...while statement is as follows:

```
do {
      Statements
} while (condition);
```

statement is executed, and then condition is evaluated.
If condition is TRUE, the loop is repeated; otherwise, the loop ends.
The statements and conditions are otherwise identical to the while loop.

****6.7. Demonstrates do...while loop.**

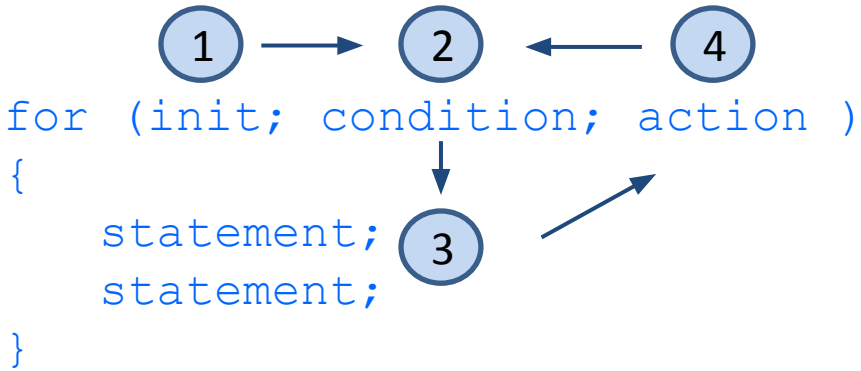
Copyright @ 2018 Real Time College

the for loop

`for` statements are powerful and flexible. includes three statements (**initialization**, **condition**, and **action**).

a `for` loop works in the following sequence:

1. Performs the operations in the initialization.
2. Evaluates the condition.
3. If the condition is `TRUE`, executes the statement(s) within the loop.
4. Do the action, and go to operation number 2



****6.9. the for loop.**

Copyright @ 2018 Real Time College

Advanced for Loops

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

After each time through, the loop repeats steps 2 and 3 of the previous page.
Multiple Initialization and Increments, It is not uncommon to initialize more than one variable, to test a compound logical expression, and to execute more than one statement.
The initialization and the action may be replaced by multiple C statements, each separated by a comma.

```
for (x=0, y=0; x<MAX_X && y<MAX_Y; x++, y++)
{
    statement;
    statement;
}
```

****6.10. multiple statements in for loops.**

****6.11. Null statements in for loops.**

****6.12. empty for loop statement.**

Copyright @ 2018 Real Time College

The “for” and while” loops differences.

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

```
float pow(float x, uint exp)
{
  float result=1.0;
  int i;
  i=0;
  while (i < exp) {
    result = result * x;
    i++;
  }
  return result;
}


int main(int argc, char **argv)
{
  float p;
  p = pow(10.0, 5);
  printf(“p = %f\n”, p);
  return 0;
}
```

```
float pow(float x, uint exp)
{
  float result=1.0;
  int i;
  for (i=0; (i < exp); i++) {
    result = result * x;
  }
  return result;
}




int main(int argc, char **argv)
{
  float p;
  p = pow(10.0, 5);
  printf(“p = %f\n”, p);
  return 0;
}
```


Copyright @ 2018 Real Time College

Nested Loops

Loops may be nested, with one loop sitting in the body of another.
The inner loop will be executed in full for every execution of the outer loop.

Note:

many C\C++ programmers use the letters `i` and `j` as counting variables.

```
for (i=0; i<MAX_I; i++)
for (j=0; j<MAX_J; j++)
{
    statement;
    ...
```

****6.14. nested for loops.**

Copyright @ 2018 Real Time College

Functions And Headers

Copyright @ 2018 Real Time College

What Is a Function?

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

A subprogram that acts on data and returns a value. Every C\C++ program has at least one function, `main()`. When your program starts, `main()` is called automatically.

`main()` might call other functions, some of which might call still others.

Each function has its own name, and when that name is encountered, the execution of the program branches to the body of that function. When the function returns, execution resumes on the next line of the calling function.

Each function should carry out a single, easily understood task. If your functions start getting large, look for places where you can divide them into component tasks.

Symbol	Addr	Value
Func_1	0xffb0	push r0
	0xffb1	push r1
	0xffb2	mov r2, r0
	0xffb3	inc r1
	0xffb4	add r2, 4
	0xffb5	**call Func_2**
	0xffb6	pop r1
	0xffb7	pop r0
	0xffb8	ret
Func_2	0xffb9	push r0
	0xffba	pop r0
	0xffbb	ret

1

2

3

Copyright @ 2018 Real Time College

What Is a Function? (cont'd)

A function can have any number of parameters, in any type we choose, or none at all.
A function can have a return value (one only) of any type, or none (void).

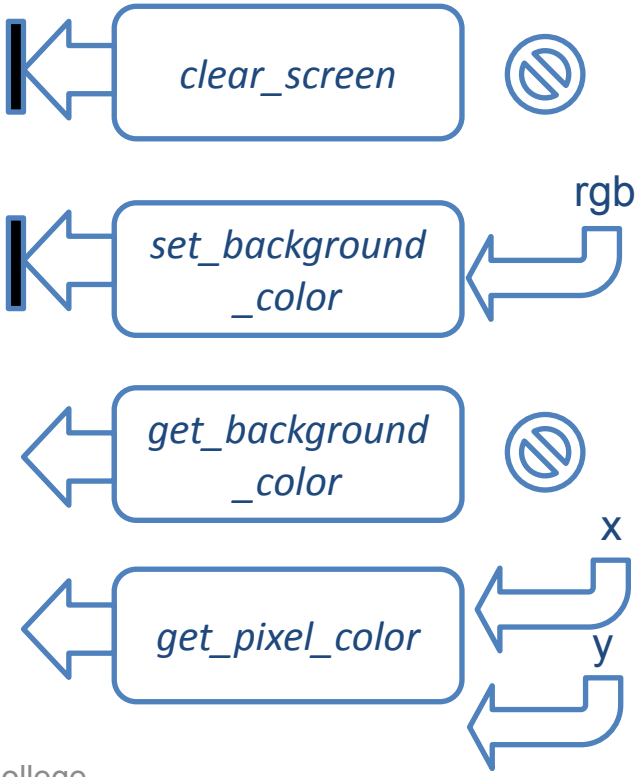
We can view a function as a black box, in which we put our raw materials and get the final value (if there should be one) in return. For example:

void clear_screen();
no parameters, no return value

void set_background_color(Color rgb);
parameter color, no return value

Color get_background_color();
no parameters, returns the color

Color get_pixel_color(int x, int y);
two parameters, returns the color



Copyright @ 2018 Real Time College

Declaring and Defining Functions

Using functions in your program requires that you :

1. declare the function first
2. define the function later.

The declaration tells the compiler the name, return type, and parameters of the function. The declaration of a function is called its prototype.

The definition tells the compiler how the function works (implementation).

Function Prototype Syntax:

```
return_type function_name ( [type [parameterName]]...);
```

Function Definition Syntax

```
return_type function_name ( [type parameterName]...)
{
    statements;
}
```

****5.1. function declaration, definition and implementation.**

Copyright @ 2018 Real Time College

Func's Example:


```
#include <stdio.h>

// declaration (aka interface / prototype)
int reverse_int(int num);
int get_right_digit(int num, int pos);
int get_left_digit(int num, int pos);

int main()
{
        int num, rev_num;
        int sec_right_digit, sec_left_digit;

        printf("please enter a number: ");
        scanf("%d", &num)

        rev_num = reverse_int(num);
        sec_right_digit = get_right_digit(num, 2);
        sec_left_digit = get_left_digit(num, 2);

        return 0;
}
```

```
// definition (aka implementation)
int reverse_int(int num)
{
        int rev = 0;

        while (rev)
        ……
        return rev;
}
int get_right_digit(int num, int pos)
{
        int digit = 0;

        while (--pos)
        ……
        return digit;
}
int get_left_digit(int num, int pos)
{
        return get_right_digit(reverse_int(num, pos);
}
```


Copyright @ 2018 Real Time College

Modular usage of functions

Usually when the code gets too long, it is recommended to divide it to multiple files.

For example, we have created a chess program, the program includes:

- Set of objects and their rules, such as the board, the pieces, and their functions.
- Printing functions, IO and GUI modules (mouse and keyboard handlers, screen etc.
- A special object which contains a list of moves done by the players
- Computer side algorithms …

and so on … it is recommended to set the modules in the following way:

Main.c

```
#include "io.h"
#include "gui.h"
#include ....

void func1();
void func2(..);

int main()
{
   ...
```

io.h

```
// declerations

int keypress(..);
int mousein(..);
..
```

io.c

```
// implementations
#include "io.h"

int keyperss(..)
{
   ...
}
int mousein(..)
{
   ...
```

gui.h

```
// declerations

int print(..);
int clear(..);
..
```

gui.c

```
// implementations
#include "gui.h"

int print(..)
{
   ...
}
int clear(..)
{
   ...
```


Copyright @ 2018 Real Time College

Interface Versus Implementation

Each function that you declare must have an implementation.
Usually the decelerations are stored in a header file - *.h (c++ uses *.hpp)
the implementations are in stored in a source file - *.c (c++ uses *.cpp)
You can think of the declarations as an interface to your functions - like a manual of how to use your code (more specifically - your functions, macros and data types).
The header file tells what data your function uses, what is its signature (param's and name) and how will they behave. For example, a header file includes the declaration:

uint8 RS232_Read(uint32 uart_address);

This means that every time we'll use *RS232_Read()* we have to put in the UART's address as a parameter, and the function will return an *uint8 value*
Whoever is using this function doesn't really care how it works (implementation).

Listing 6.5. A demonstration of violations of the interface.

Copyright @ 2018 Real Time College

Declarations and Definitions (example)

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

main.c

```
#include "io.h"
#include "gui.h"
#include ....

void init ();
void showMenu();

int main()
{
   char buffer[120], key;
   ..

   init ();
   memset (buffer, 0, sizeof (buffer) );
   ..

   do {
      showMenu() ;
      key = keypress();
      ..
   }while (stat != EXIT || keypress() != ESC);

   return 0;
}

void init ()
{
   ..
```

io.h

```
// declerations

#ifndef IO_H
#define IO_H

#define ESC 27

int keypress();
int mousein(..);
..
#endif
```

#ifdef / #ifndef / #endif directives:

Done to prevent re-declaration of types, functions variables and constants.

Also useful for adding debugging information and multi-arch support.

io.c

```
// implementations
#include "io.h"

// inputs a key press from the keyboard, 0 for no input
char keyperss()
{
   ...
}

// imputs mouse's status ....
int mousein(..)
{
   ...
```


Copyright @ 2018 Real Time College

Variables Scope

Every Variable is Defined within some scope.

A Variable's name and a function's name are also called symbols

Lexical scopes are defined with curly braces { }. And defines a local area.

The scope written after a Function is the complete body of the function.

Variables defined inside the body of the function are "Local" and only from within that scope they can be accessed.

Variables defined outside the function are Called "Global" and their scope is the hole program .

(Returns nothing)

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

```
void p(char x)
{
                /* p,x */
  char y;
                /* p,x,y */
  char z;
                /* p,x,y,z */
}
                /* p */
char z;
                /* p,z */

void q(char a)
{
  char b;
                /* p,z,q,a,b */
  {
    char c;
                /* p,z,q,a,b,c */
  }

  char d;
  /* p,z,q,a,b,d (not c) */
}

/* p,z,q */
```


Copyright @ 2018 Real Time College

Local Variables

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

you can declare **variables within the body of the function**. These variables are called local variables, because they exist only locally within the function itself. When the function returns, the local variables are no longer available **(only on the Stack)** .

Local Variables are visible and usable only within the function in which they are defined.

In C you should define variables only at the beginning of the function. In C++ you can define variables anywhere within the function, not just at its top.

The scope of the variable is the block in which it is defined.

The parameters passed in to the function are also considered local variables

****5.2. local variables and parameters.**

Copyright @ 2018 Real Time College

Local Variables

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

```
#include <stdio.h>
#include <inttypes.h>

int add()
{
  int a=5;
  int b=6;
  int sum;
  Int mul;

  Sum=a+b;
  Mul=a*b
  return sum + mul;
}
```

```
int main(int argc, char **argv)
{
  int ret = 0;
  add();
  return ret;
}
```

The Stack in main(..)

int ret	0
int argc	?
char **argv	?

The Stack in add()

int ret	0
int argc	
int **argv	?
int a	5
int b	6
int sum	0
int mul	0

****5.2. The use of local variables and parameters.**

Copyright @ 2018 Real Time College

Global Variables

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Variables defined **outside of any function have global scope and are available from any function in the program**, including `main()`.

Local variables with the same name as global variables do not change the global variables, the Local variable hides the global variable. Reference to the same variable name within the function will lead to the local variable--not the global one.

	Stack Segment		Data Segment	
main()	`int argc`	?	`int sum`	0
	`char **argv`	?		
add_m()	`int sum1`	0		
	`int a`	1		
	And so on..			

```
#include <stdio.h>
#include <inttypes.h>

int sum=0;
int add_m(int sum1)
{
  int a=1;
  int b=2;
  int mul;
  mul=a*b;
  sum =sum1+Mul
  return sum;
}

int main(int argc, char **argv)
{
  add_m(sum);
  return 0;
}
```

****5.3. global and local variables.**

Copyright @ 2018 Real Time College

Global Variable scopes

In C\C++, global variables are legal, but they are almost never used.
Global variables are a dangerous but necessary tool. They are necessary because there are times when the programmer needs to make data available to many functions and he does not want to pass that data as a parameter from function to function.
Global's are dangerous because they are shared data, and one function can change a global variable in a way that is invisible to another function. This can and does create bugs that are very difficult to find.

****5.4 – variables scoped within a block**

Copyright @ 2018 Real Time College

By-Ref Functions

In most programming language, functions' parameters are able to be configured in two ways.

By Value – copies the value to the parameter (which duplicate its value to the parameter of the function). Changes to this parameter **will not** affect the original variable.

By Reference – works on the original variable itself (which copies the address of the variable to the parameter of the function). Changes **will** affect the given variable.

In C Programming, passing By-Ref parameters is not supported. It is done in another way. The only way is to send the address's value (form of by-val):

```
int main()
{
    int a=5;
    inc(&a);
}
```

```
void inc(int *x)
{
   (*x)=(*x) + 1;
}
```


Copyright @ 2018 Real Time College

byRef Functions (c++ only)

C++ enables you to send a reference parameter in functions.
The reference copies the address of the symbol when the function is called. Then uses it from within the function (as if the name of the variable is simply changed) .

```
void myFunction (int &);
int main()
{
    int c=10;
    printf("c=%d\n", c); //output: c=10
    myFunction(c);
    printf("c++=%d\n", c); // output: c++=11
}
void myFunction (int &ref)
{
    ref++;
}
```

```
// swap in C
void swap(int *a1, int *a2);

int main()
{
   int x=7, y=11;
   swap(&x, &y);
   printf("x=%d, y=%d, x, y);

   return 0;
}

void swap(int *a1, int *a2);
{
   int temp;

   temp=*a1;
   *a1=*a2;
   *a2=temp;
}
```

```
// swap in C++
void swap(int &a1, int &a2);

int main()
{
   int x=7, y=11;
   swap(x, y);
   printf("x=%d, y=%d, x, y);

   return 0;
}

void swap(int &a1, int &a2);
{
   int temp;

   temp=a1;
   a1=a2;
   a2=temp;
}
```

//both outputs: x=11, y=7

Copyright @ 2018 Real Time College

Default Parameters (c++ only)

For every parameter you declare in a function prototype and definition, the calling function must pass in a value. The value passed in must be of the declared type. Thus, if you have a function declared as

long myFunction(int);

If the function definition differs, or if you fail to pass in an integer, you will get a compiler error. The one exception to this rule is if the function prototype declares a default value for the parameter.

default value is a value to use if none is supplied.

The preceding declaration could be rewritten as

long myFunction (int x = 50);

****5.7 – use of default parameter values**

Copyright @ 2018 Real Time College

Overloading Functions (c++ only)

C++ enables you to create more than one function with the same name.
This is called **function overloading** (function *polymorphism*).
The functions must differ in their parameter list, with a different type of parameter, a different number of parameters, or both.

```
int myFunction (int, int);
int myFunction (long, long);
int myFunction (long);
```

The return types can be the same or different on overloaded functions.

****5.8. A demonstration of function polymorphism.**

Copyright @ 2018 Real Time College

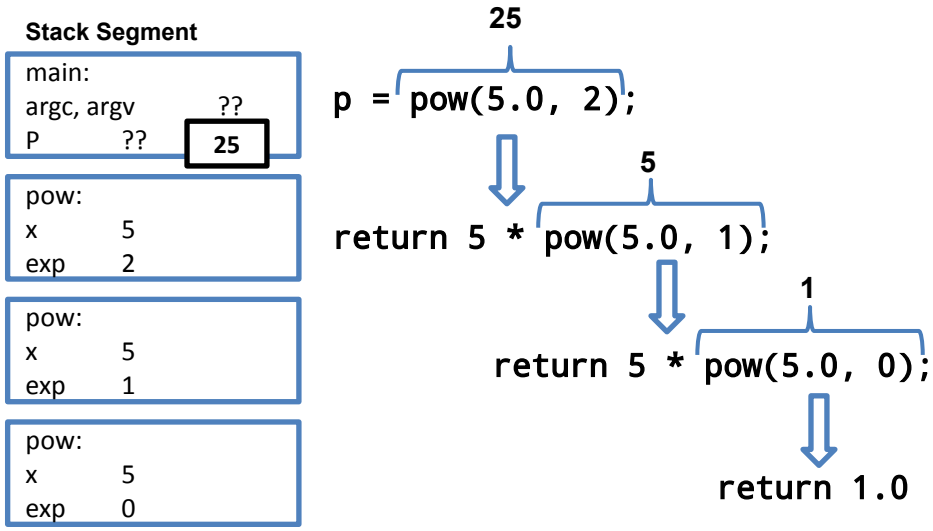
Recursion

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

If a variable is valid "within the scope of a function", what happens when you call that function recursively? Is there more than one "exp"?

Yes. Each function call allocates a "stack frame" where Variables within that function's scope will reside.

```
#include <stdio.h>
#include <inttypes.h>

float pow(float x, uint32_t exp)
{
  /* base case */
  if (exp == 0) {
    return 1.0;
  }

  /* "recursive" case */
  return x*pow(x, exp - 1);
}

int main(int argc, char **argv)
{
  float p;
  p = pow(5.0, 2);
  printf("p = %f\n", p);
  return 0;
}
```



Copyright @ 2018 Real Time College

Recursion

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Problem: “recursion” eats stack space (in C).
Each loop must allocate space for arguments and local variables, because each new call creates a new “scope”.

Solution: “while” loop.

```
loop:
  if (condition) {
    statements;
    goto loop;
  }
```

⇨

```
while (condition) {
  statements;
}
```

```
float pow(float x, uint exp)
{
  int i=0;
  float result=1.0;
  while (i < exp) {
    result = result * x;
    i++;
  }
  return result;
}

int main(int argc, char **argv)
{
  float p;
  p = pow(10.0, 5);
  printf(“p = %f\n”, p);
  return 0;
}
```

****5.10. Demonstrates recursion using the Fibonacci series.**

Copyright @ 2018 Real Time College

Arrays

Copyright @ 2018 Real Time College

What Is an Array (1)?

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

An array is a collection of data storage locations, each of which holds the same type of data. Each storage location is called an element of the array.

For instance

```
long LongArray[25];
```

declares an array of 25 elements (type `long`), named `LongArray`.

this sets aside enough memory to hold all 25 elements (4 bytes *25=100 bytes) of continuous memory.

Applying sizeof to an array name yields the size of the whole array, in bytes.

	Addr	Element
LongArray[0]	0xf0	1st
LongArray[1]	0xf4	2nd
LongArray[2]	0xf8	3th
LongArray[23]	?	24th
LongArray[24]	?	25th

Copyright @ 2018 Real Time College

What Is an Array (2)?

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Referring to LongArray is a reference to an address in memory (similar to any variable),
However, `LongArray` is an address, not a value of a type. Which means:

`LongArray` equals `&LongArray` which is an address in memory (0xf0…)

You can print that address by printing it:

```
Printf("%p", LongArray);
```

You can also print an address of a specific element:

```
Printf("%p", &LongArray[2]);
```

	Addr	Element
LongArray[0]	0xf0	1st
LongArray[1]	0xf4	2nd
LongArray[2]	0xf8	3th
LongArray[23]	?	24th
LongArray[24]	?	25th

Copyright @ 2018 Real Time College

What Is an Array(3)?

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

You access each of the array elements by using their index.
the first array element is `LongArray[0]`, `LongArray[1]` is the second, and so on.

long LongArray[25]; // declaration
For this declaration the existing elements are *LongArray[0] - LongArray[24]*,
LongArray[25] – DOESN'T EXIST!

This is something you need to be aware of,
C will let you access that element !!

	Addr	Element
LongArray[0]	0xf0	1st
LongArray[1]	0xf4	2nd
LongArray[2]	0xf8	3th
LongArray[23]	?	24th
LongArray[24]	?	25th

**8.1. Using an integer array.

Copyright @ 2018 Real Time College

What Is an Array(4)?

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

NOTE:

Arrays count from 0, not from 1. This is the cause of many bugs in programs.

Whenever you use an array, remember that an array with 10 elements counts from ArrayName[0] to ArrayName[9]. There is no ArrayName[10].

However the compiler makes a simple offset calculation to get to the desired index, and can easily be set with the incorrect value.

```
#include <stdio.h>
#define MAX 25

Int main() {
        long LongArray[MAX];
        LongArray[MAX-1] = 100; // last element
}
```

	Addr	Element
LongArray[0]	0xf0	1st
LongArray[1]	0xf4	2nd
LongArray[2]	0xf8	3th
LongArray[23]	?	24th
LongArray[24]	?	25th

Copyright @ 2018 Real Time College

Writing Past the End of an Array

Lets take for instance this declaration:

```
long LongArray[25];
```

It has 25 elements, Writing to LongArray[5], which is the sixth element. The compiler multiplies the offset (5) by the size of each element in this case 4.
so it writes to this offset (20) from the beginning of the array.

What if you write to `LongArray[50]`, the compiler ignores the fact that there is no such element. It computes the location (200 bytes) and then writes over whatever is at that location. This might have unpredictable results. If you're lucky, your program will crash immediately, if not it'll take a long time to find the BUG!!

It is so common to write to one past the end of an array that this bug has its own name. It is called a **fence post error**, (how many fence posts you need for a 10 meter Fence – 9 posts).

**12.2. Writing past the end of an array.

Copyright @ 2018 Real Time College

Initializing Arrays

After the array name, you put an equal sign (=) and a list of comma-separated values enclosed in braces.

```
int IntegerArray[5] = { 10, 20, 30, 40, 50 };
```

declares an Integer Array to be an array of five integers. It assigns IntegerArray[0] =10, IntegerArray[1] = 20, and so forth.

```
int IntegerArray[5] = { 10, 20, 30, 40, 50, 60};
```

generates a compiler error, 5 elements declared and 6 initialized.

```
int IntegerArray[5] = { 10, 20};
```

uninitialized array members will be initialized to `0`.

```
int IntegerArray[] = { 10, 20, 30, 40, 50 };
```

exactly the same array as you did in the previous example.

**12.3. Using consts and enums in arrays.

```
Num_of_elements =
sizeof(IntegerArray)/sizeof(IntegerArray[0]);
```


Copyright @ 2018 Real Time College

Multidimensional Arrays

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

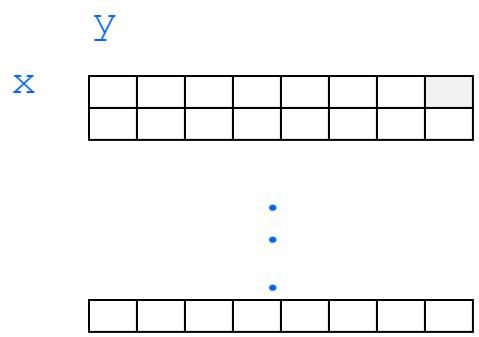
It is possible to have arrays of more than one dimension.
Arrays can have any number of dimensions, although most of the arrays you create will be of one or two dimensions.

A good example of a two-dimensional array is a chess board.
One dimension represents the eight rows; the other dimension represents the eight columns.

```
char Board[8][8];
```

Rows columns

x y



**8.5. Creating a multidimensional array..

Copyright @ 2018 Real Time College

Pointers

Copyright @ 2018 Real Time College

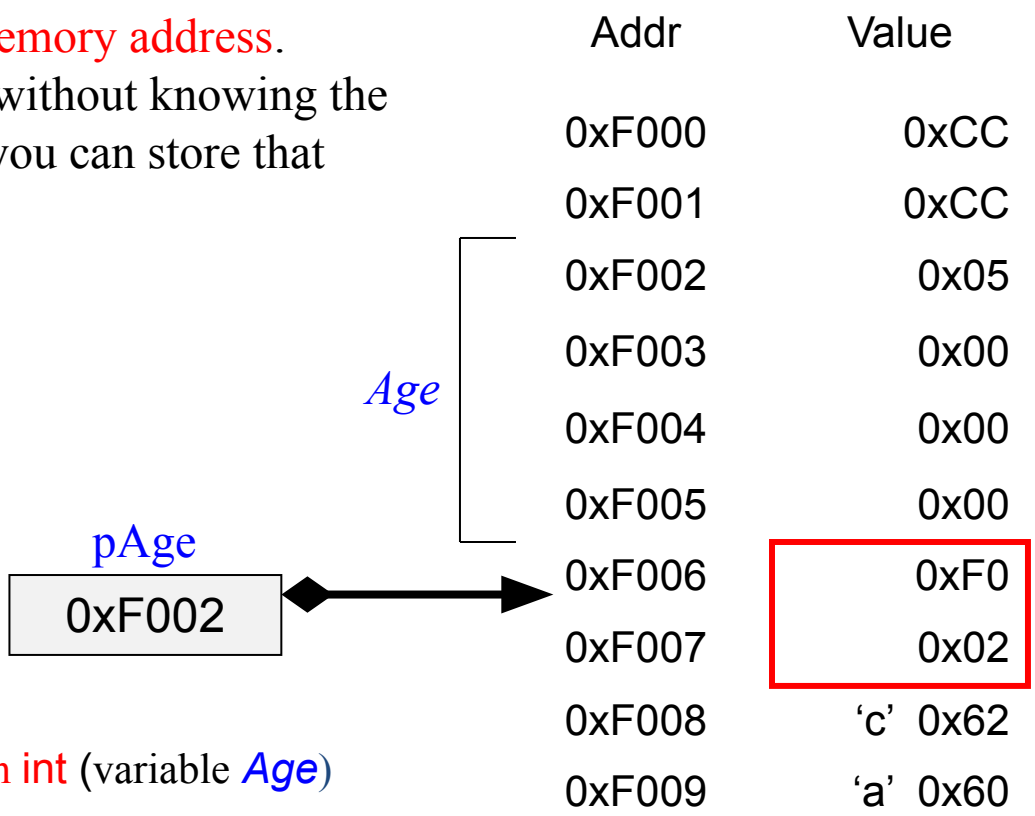
What is a Pointers

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

A *pointer* is a variable that holds a memory address. Every variable has an address. Even without knowing the specific address of a given variable, you can store that address in a pointer.

int Age = 5;
*int *pAge = NULL;*
pAge = & Age; => pAge= 0xf002
**pAge = 5*

* - pointer
& - address of



pAge is declared to hold the address of an int (variable *Age*)

When pointers are created, they should be initialized to something. If you don't know what you should assign, assign 0 , this is a **NULL** pointer.

8.1. Address of variables.

Copyright @ 2018 Real Time College

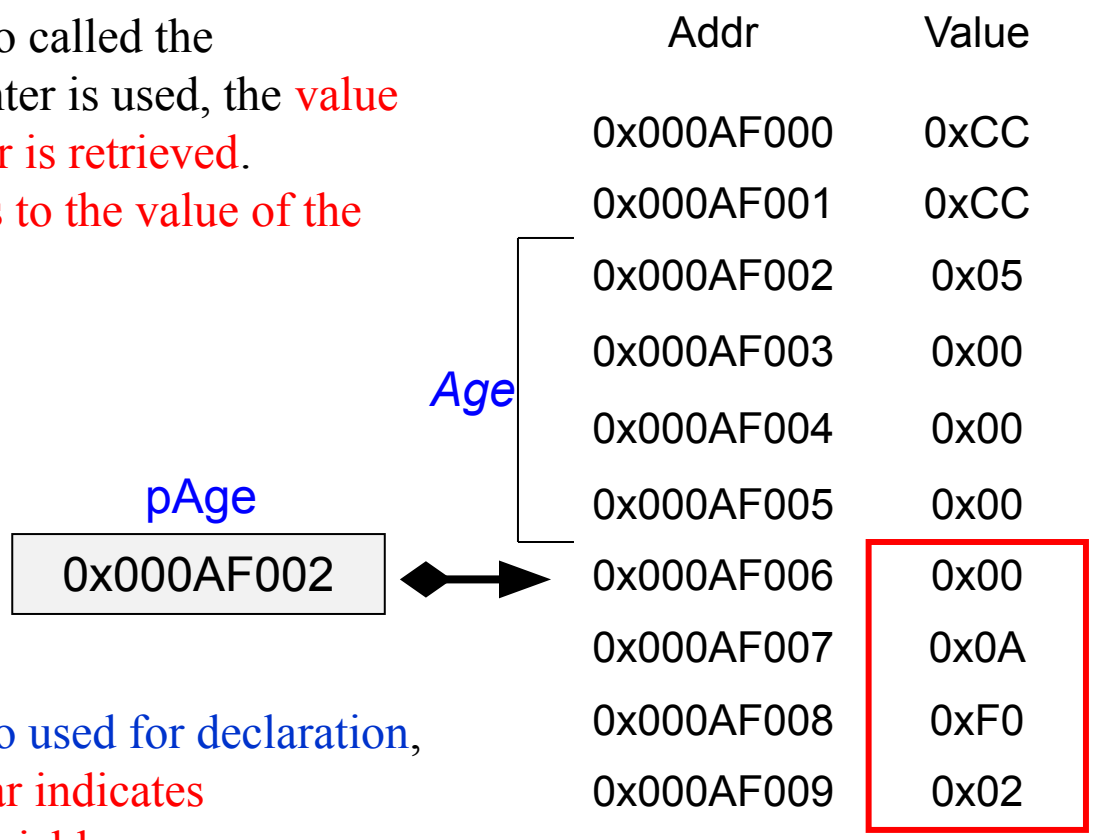
The Indirection Operator

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

The indirection operator (*) is also called the dereference operator. When a pointer is used, the value at the address stored by the pointer is retrieved.

A pointer provides indirect access to the value of the variable whose address it stores.

int Age = 5;
*int *pAge = 0; // 0 = NULL*
pAge = & Age;
Age= 5
**pAge = 5*



The indirection operator (*) is also used for declaration, When a pointer is declared, the star indicates that it is a pointer, not a normal variable.

*int *pAge=0;*

8.2. create a pointer.

Copyright @ 2018 Real Time College

Using pointers

Declare a Pointer: write the type of the variable or object whose address will be stored in the pointer, followed by the pointer operator (*) and the name of the pointer.

```
int * pVar1;
int * pVar2 = NULL;     // NULL equals to 0 -- #define NULL 0
```

Assign or initialize a pointer: use the name of the variable whose address is being assigned with the 'address of' operator (&).

```
int iVar = 5,  *pVar1 = &iVar,  *pVar2;   // declaration, the * must be there..
pVar2 = &iVar;                            // usage, here we set an addr'. no*
/* notice the difference between pVar1 and pVar2 assignment */
```

Dereference a pointer: use the pointer name with the dereference operator (*) :

```
int iVar = 5, *pVar1 = &iVar;             // declaration   (of pVar1)
int theValue = *pVar1, *pVar2 = pVar1;    // usage         (of pVar1)

*pVar2=*pVar1
```

8.3. Finding out what is stored in pointers.

Copyright @ 2018 Real Time College

By-ref parameters

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

```
void Func (int byVal, int *byRef);

void main()
{
        int a = 5, b = 9;
        Func(a, &b);
}

void Func (int byVal, int *byRef)
{
        byVal++;
        (*byRef)++;
}
```

Before Func execution

main()	a: 0x1200440	5
	b: 0x1200444	9

In Func execution

Func()	byVal: 0x1201200	5
	byRef: 0x1201204	0x1200444

After Func execution

main()	a: 0x1200440	5
	b: 0x1200444	10

Copyright @ 2018 Real Time College

Invalid pointers

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

A pointer to a variable allocated on the stack becomes invalid when that variable goes out of scope and the stack frame is "popped".
The pointer will point to an area of the memory that may later get reused and rewritten

```
char * get_pointer()
{
  char x=0;
  return &x;
}

int main()
{
  char *ptr = get_pointer();
  *ptr = 12;  /* valid? */
  other_function();
}
```

ptr points to a location that's no longer in use, and will be reused the next time a function is called!

A Valid pointer is one that points to memory that your program controls.
Using invalid pointers will cause non-deterministic behavior, and will often cause the Operating system (Linux) to kill your process (SEGV or Segmentation Fault) if you are lucky.

Copyright @ 2018 Real Time College

Strings

Copyright @ 2018 Real Time College

Strings (1)

- Strings in C are defined as arrays of characters.
  For example, the following defines a string of 50 characters:
  char name[50];

- C has no string handling facilities built in. This means that intuitive operations such as:

```
char firstname[50],lastname[50],fullname[100];
firstname= "Arnold"; /* Illegal */
lastname= "Schwarznegger"; /* Illegal */
fullname= "Mr"+firstname +lastname; /* Illegal */
```

- However, there's a special library of strings handling routines <string.h>
  with which such operations are done.

Copyright @ 2018 Real Time College

Strings (2)

- Assigning the \0 character to some item in the array turns it from an ordinary array of chars to a C string, so that it's ready for use of string.h routines.

- \0 is used to indicate the end of string (not the memory allocation). If we store the word "DAVE" in it, its contents will look like:

D	A	V	E	\0			

This means that in terms of strings, the length of that string is 4, not including the null, although the memory it occupies is of 50 chars.

Length size is achieved by using the *strlen() function*

Copyright @ 2018 Real Time College

Strings Initializations

1. *char name [7] = “DavidG“;*
   Typically, a compiler sets aside an area in memory (string and text literals area) to hold all the quoted strings used in the program source code, associating each stored string with its address. In our case, this enables name to serve as another synonym to the address where "DavidG" lies, (just as DavidG is a pointer to that address).

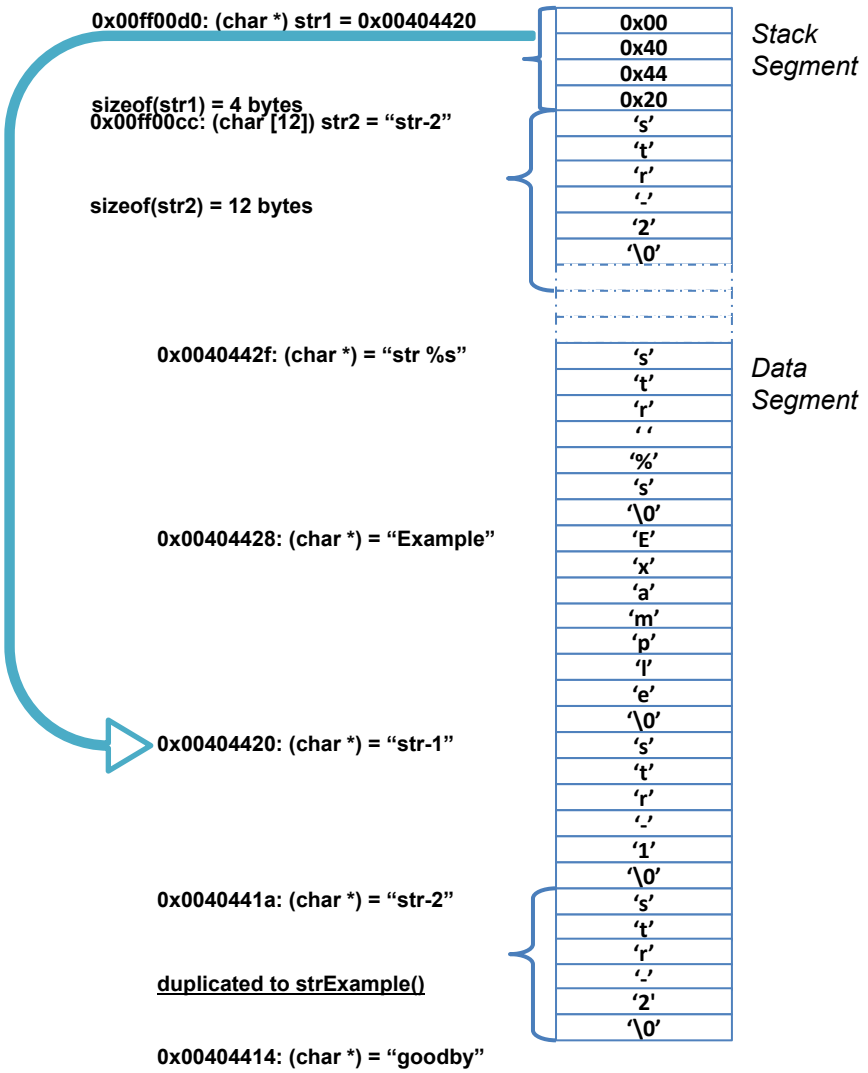
1. *char name [] = “DavidG“;*
2. *char name [] = {‘D’, ‘a’, ‘v’, ‘i’, ‘d’, 'G', ’\0’};*
3. *char* name = “DavidG”; //?? Is it really allowed*

Copyright @ 2018 Real Time College

Strings Initializations

```
#include <stdio.h>
#include <string.h>

void strExample();

int main()
{
        printf("str %s", "Example");
        strExample();
        printf("goodbye");

        return 0;
}

void strExample()
{
        char *str1 = "str-1";
        char str2[12] = "str-2";
}
```

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ



Copyright @ 2018 Real Time College

Const for Pointers

What's the difference?

Pointers to arrays : char x[ ] vs. char* x

- *<type>[ ]* belongs to the category of char* const ptr.
- char name* = " abc..." belongs to the category of const char* ptr .

- **char* const ptr** - ptr is a pointer to a constant **address**. ptr isn't allowed to point to any other address.
- **const char* ptr** - ptr currently points to a constant **content**, which is not allowed to be modified. However, ptr is allowed to point to somewhere, .
- **const char* const ptr** - ptr is a pointer to a constant **address**, and the content in that address is constant.

Copyright @ 2018 Real Time College

Pointers to arrays

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

What's the difference?

Pointers to arrays : char x[ ] vs. char* x

- *x* is of type char[]. Therefore, ptr can't be changed and operators attempting to do so, such as: *x*++ or *x*-- are illegal.

- char* x is much the same, only that *sizeof(name) the number of elements,* is decided by the programmer rather than letting the compiler do that automatically as is on the first case.

- The third initialization causes *name* to be a pointer of type const char* ptr meaning that name can be assigned to point at other addresses, but this current address is unchangeable. The memory space is allotted and written once, when initialized, and further changes to it are illegal:

```
const char* name = "DavidG";   //recommended usage of const
name[2] = 'd';                 //illegal. No change permitted
name++;                        //legal, but cautious
strcpy(name,"gameover");       //buffer overflow,
strcpy                         //must use strings with finite length
```


Copyright @ 2018 Real Time College

(char), (char *), (char[])

If you are having a little problems putting it all together … let's practice:

```
char c = 'a', arr[5] = "abcd", *p = arr;
```

- The variable c is type: _____ and to print it write: printf("__", c);
- The variable arr is type: _____.

–Printing a char in the array: printf("___", ___); and the printed type is: _____
–Printing the array (string assumed): printf("___", ___); and the printed type is: _____
–According to the way the array was defined it is positive that arr[4] = __
–What will be printed by that? printf("%s %s %s", arr, arr+2, &arr[2]);
–Will the result of printing arr 's address is the same as printing &arr ? ___. Do it!
–What will be the result of trying to compile the next line: arr++;

_____

- The variable p is type: _____ . To print that pointer: printf("__", _____);

–Printing a character using p: printf("___", ___); and the printed expression's type _____
–Will the result of printing p 's address is the same as printing &ch_ptr ? ___. Do it !
–Printing p as defined earlier is ____'s address. printing &p is ____'s address.
–What will be the result of trying to compile the next line: p++;
–What will be the result of printf("%s", p); after p++;

Copyright @ 2018 Real Time College

Useful string.h routines

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Gets the string length

```
size_t strlen ( const char* str );
```

Returns the length of *str*: the amount of characters between the beginning of the string and the terminating null character.

Copy strings

```
char* strcpy( char* destination, const char* source );
```

Copies the C string pointed by *source* into the array pointed by *destination*, including the terminating null character.

Notes:

To avoid overflows, the size of the array pointed by *destination* shall be long enough to contain the same C string as *source* (including the terminating null character), and should not overlap in memory with *source*.

Related functions:

```
char* strncpy ( char* dest, const char* source, size_t num);
```

similar to strcpy but copies only the first **num** characters of **source** to **dest**.

Does not automatically add '\0' at the end! However, if **strlen(source) < num**, the rest of **dest** is filled with '\0' until num characters are written (**num - strlen(source)**)

****strlen_examp.cpp**

Copyright @ 2018 Real Time College

Working Strings on bytes resolution

There's the option to manipulate all the above routines, copying, concatenating and comparing straightforward on the raw bytes those strings are pointing to **Fill block of memory**

void memset (void* ptr, int value, size_t num);*

Sets the first *num* bytes of the block of memory pointed by *ptr* to the specified *value* (interpreted as an unsigned char).

Example

```
#include <stdio.h>
#include <string.h>
int main ()
{
        char str[] = "almost every programmer should know memset!";
        memset (str,'-',6);
        puts (str);          // well, don't panic .. puts(..) is the same as printf("%s" ..)
        return 0;
}
//output: ------ every programmer should know memset!
```


Copyright @ 2018 Real Time College

Strings on bytes resolution-cont'd

Copy block of memory

void memcpy (void* dest,const void* source, size_t num);*

Notes:

- Copies the values of *num* bytes from the location pointed by *source* directly to the memory block pointed by *destination*.
- binary copy of the data: type of the objects are irrelevant
- Always copies exactly *num* bytes. Beware of data overflows!
- for overlapping memory blocks, *memmove()* is a safer approach. It does the same but copying takes place as if an intermediate buffer was used, allowing the *dest* and *source* to overlap.

****memcpy_examp.cpp**

Copyright @ 2018 Real Time College

Strings on bytes resolution-cont'd

Compare two blocks of memory

- *int memcmp (const void* ptr1, const void* ptr2, size_t num);*

Similar to strcmp, doing so directly byte by byte. See references for further details.

And Many More…

- *atoi* - Convert string to integer

  #include <ctype.h>
- *tolower/upper* – turns a given character to lower/uppercase letters
- *isalpha* - Check if character is alphabetic
- *isdigit* - Check if a character is a decimal digit

You're invited to try implementing some of them by yourself …

Copyright @ 2018 Real Time College

Dynamic Memory Allocation

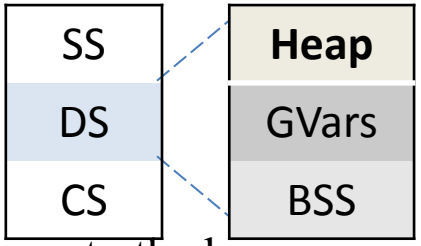
Copyright @ 2018 Real Time College

The Free Store (Heap)

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

The Memory is divided into 5 main areas:

- Registers - Used for internal housekeeping functions, such as keeping track of the top of the stack and the instruction pointer
- Stack Seg (SS) - Local variables, along with function parameters.
- Data Seg (DS) - Global variables, But contains more segments.
- Code Seg (CS) - Where the Code is run from
- The free store (aka Heap) - Just about all remaining memory is given to the heap



A problem we might run into without the Heap…

Local variables are valid as long as you are in the defining function, when the function returns, the local variables are thrown away.

Global variables solve that problem at the cost of unrestricted access throughout the program, which leads to a difficult code to understand and maintain.

Putting data in the free store solves both of these problems.

Copyright @ 2018 Real Time College

Why Do we need the Heap?

Suppose you want to create a String Array representing someone's last name. The simplest way is to use a character array member to hold the name.
But this has some drawbacks. You might use a 14-character array and then run into 'Bartholomew Smeadsbury-Crafthovingham'. Or, to be safer, you may use a 40-character array.
But, if you then create an array of 2000 such Strings, you'll waste a lot of memory with character arrays that are only partly filled. (At this point, we're adding to the computer's memory load.)
There is an alternative. Often it is much better to decide many matters, such as how much storage to use, in Real Time when a program is running
The usual approach is to make a dynamic allocation for the correct amount of memory while the program is running.
The allocation\de-allocation could be quite tricky at times (why?)

Copyright @ 2018 Real Time College

The Free Store (Heap)

The stack is cleaned automatically when a function returns. All the local variables go out of scope, and they are removed from the stack.
The free store is not cleaned until your program ends, and it is your responsibility to free any memory that you've reserved when you are done with it.

- The memory you reserve remains available until you explicitly free it.

- If you reserve memory on the free store while in a function, the memory is still available when the function returns.

- Accessing memory in this way, rather than using global variables, is that only functions with access to the pointer have access to the data. This provides a tightly controlled interface to that data, and it eliminates the problem of one function changing that data in unexpected and unanticipated ways.

- For this to work, you must be able to create a pointer to an area on the free store and to pass that pointer among functions.

Copyright @ 2018 Real Time College

malloc

- Allocates memory block on the free store, returning a pointer to the beginning of the block.

- If *malloc()* cannot create memory on the free store (memory is, after all, a limited resource) it returns the null pointer.

```
#include<stdlib.h>
void * malloc (size_t size);
```

(`size` - Size of the memory block, in bytes.)

- The content of the newly allocated block of memory is not initialized, remaining with indeterminate values.

- To create an unsigned short variable on the heap:

```
char *ptr = malloc(sizeof (char));
```

ptr - return value from *malloc* is the memory address and It must be cased to the pointers type.

Copyright @ 2018 Real Time College

malloc (continued)

To create an int variable on the free store, you might write

```
int *pPointer;
pPointer = malloc(sizeof(int));
if (NULL == pPointer)
{
    Printf("error allocating Memory");
    return 1;
}

*pPointer = 72;
```

This means, "Put 72 to the address on the free store to which pPointer points."
Always check your pointer for NULL.

If *malloc()* cannot create memory on the free store let the programmer know and act accordingly..

Copyright @ 2018 Real Time College

realloc

To create a dynamic array, simply allocate N elements * sizeof(type) that you need.

```
int *pPointer, N=5;
pPointer = malloc(sizeof(int));
if (NULL == pPointer)
{
    Printf("error allocating Memory");
    return  (-1);
}

pPointer = realloc(pPointer, N*sizeof(int));
...
```

Using a pointer is basically working with an address, same as working with arrays. Dynamically allocated arrays are treated the same way as regular arrays.

Copyright @ 2018 Real Time College

Free (void *ptr)

- When you are finished with your area of memory, you must call free on the pointer.
- free de-allocates (returns) the memory to the free store.

```
#include <stdlib.h>
void    free(void *ptr);
```

Notes:

- the pointer itself is a local variable. When the function in which it is declared returns, that pointer goes out of scope and is lost.
- The memory allocated with malloc is not freed automatically, if That memory becomes unavailable a situation called a memory leak occurs. It's called a memory leak because that memory can't be recovered until the program ends.

To restore the memory to the free store, you use the keyword free system call.

```
free (pPointer);
pPointer=NULL; //always zero it
```

** 8.5 allocating variables on the heap

Copyright @ 2018 Real Time College

Pointer calculations

What happens if we add 1 to a pointer that points to a variable of some type ?

```
int *pHeap = NULL;
pHeap = malloc(sizeof(int * 10));
*(pHeap+1) // increased by 1 byte?? NO it increases by 4bytes (=sizeof(int))
```

The pointer is incremented by the size of the variable type it points to.

Copyright @ 2018 Real Time College

malloc \ free – Warnings

Each time you allocate memory you must check to make sure the pointer is not null) there was enough memory in the pool to allocate).

Stray /Dangling pointers :

1. When you delete a pointer, the memory it points to is freed. Calling delete on that pointer again will crash your program! When you delete a pointer, set it to zero (null).

2. Using a pointer after freeing it without nullizing it, will the program to be unstable (or crash the program). Calling delete on a null pointer is guaranteed to be safe.

Memory leaks :

3. For every time in your program that you allocate memory, there should be a call to de-allocate it.

4. Use caution when allocating memory inside a function and exiting that function freeing the allocated memory

```
char  *pChar = malloc(sizeof(char));
free (pChar) ;     //frees the memory
pChar = 0;      //sets pointer to null
```


Copyright @ 2018 Real Time College

Memory Leaks

Another way you might inadvertently create a memory leak is by reassigning your pointer before deleting the memory to which it points. Consider this code fragment:

```
int *ptr = malloc(sizeof(int));
 *ptr = 72;
 ptr = malloc(sizeof(int)); //new address!! Memory Leak!
* ptr = 84;
```

The code should have been written like this:

```
int *ptr = malloc(sizeof(int));
 *ptr = 72;
free(ptr);
  ptr = malloc(sizeof(int)); // allocate new Memory space from O.S.
 *ptr = 84;
```


Copyright @ 2018 Real Time College

Arrays on the Free Store

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Nothing new here use malloc, sizeof and **casting** as usual

*pHeap = malloc(sizeof(int * 10));* //pHeap points to 1st Element

declares pHeap to be a pointer to an array on the heap.

In order to access Array's elements on the heap we increment the pointer (the Compiler know how to calculate the offset)

```
*pHeap = 10;          // set 1st Element to 10
 pHeap ++;            // advance to 2nd Element
 *(pHeap +2) = 20;    // set 4th Element to 20
```

** 8.6 allocating an array on the heap

129

Copyright @ 2018 Real Time College

Deleting Arrays on the Free Store

In order to delete the array on the heap we use free as usual:

```
free (pHeap );
pHeap = NULL;
```

this returns all the memory set aside for the array back to O.S., and forbids from Stray /Dangling pointers

130
Copyright @ 2018 Real Time College

Stray or Dangling Pointers

One source of bugs that are nasty and difficult to find is stray pointers.
A stray pointer is created when you call free on a pointer-thereby freeing the memory that it points to, and later try to use that pointer again without reassigning it.

be careful not to use a pointer after you have called delete on it. The pointer still points to the old area of memory, but the compiler is free to put other data there; using the pointer can cause your program to crash. Worse, your program might proceed merrily on its way and crash several minutes later.

To be safe, after you delete a pointer, set it to null (0).

****8.4. Creating a stray pointer.**

Copyright @ 2018 Real Time College

Dynamic Memory-Sum it up (Heap)

Dynamic memory is useful.
Part of C strategy is letting the program decide about memory during runtime rather than during compile time. That way, memory use can depend on the needs of a program instead of upon a rigid set of storage-class rules. Dynamic memory has several caveats:

- Whereas the stack is automatically reclaimed, **dynamic allocations must be tracked and free()'d when they are no longer needed**. With every allocation, be sure to plan how that memory will get freed. Losing track of memory is called a "memory leak".

  Reference counting

- Whereas the compiler enforces that reclaimed stack space can no longer be reached, **it is easy to accidentally keep a pointer to dynamic memory that has been freed**. Whenever you free memory you must be certain that you will not try to use it again. **It is safest to erase any pointers to it**.

- Because dynamic memory always uses pointers, there is generally no way for the compiler to statically verify usage of dynamic memory. This means that **errors that are detectable with static allocation are not with dynamic**

Copyright @ 2018 Real Time College

Arguments to main

Copyright @ 2018 Real Time College

argc & argv what are they ?

The declaration of main looks like this:

*int main(int argc, char *argv[]);*

There are two arguments to main: argc and argv.

argc **-** is a count of the arguments supplied to the program.
argv - is an array of pointers to strings.

These arguments are passed to the program by the host system's command line interpreter

you will often see the declaration of main expressed in these terms as well:

*int main(int argc, char **argv);*

Copyright @ 2018 Real Time College

argc & argv what are they ?

When a program starts, the arguments to main will have been initialized to meet the following conditions:

argc is greater than zero.
argv[argc] is a null pointer.
argv[0] through to argv[argc-1] are pointers to strings whose meaning will be determined by the program.
argv[0] will be a string containing the program's name or a null string if that is not available.

Remaining elements of argv represent the arguments supplied to the program. In cases where there is only support for single-case characters, the contents of these strings will be supplied to the program in lower-case.

Copyright @ 2018 Real Time College

Pointers to Functions

Copyright @ 2018 Real Time College

Pointers to Functions (1)

- A function's name is actually a const pointer to that function.

- When alternative functions may be used to perform similar tasks on data. You can pass the data and the function to be used to some ***control*** function for instance.
  Declaration:
  1. *long (* pFunc)(int);*
  2. *long* Func (int);*

  Question: What's the difference?

  The second declaration –Func is a function which gets an int and returns a pointer to type long variable.
  The first declaration- pFunc is a pointer to function which gets an int and returns a type long result.

Copyright @ 2018 Real Time College

Pointers to Functions (2)

- **long (*)(int)**is a type just like any other types: int*, char, enum color* .

pFunc is a pointer of such type.

If it's too long for you, replace it by typedef in this manner:

*typedef long(*LPF)(int, int); // the type is shortly called LPF*

Invoking the function by the pointer, Both are equivalent.

- *pFunc(x);*
- *(*pfunc)(x);*

We can use the pointer to invoke a specific function once it's assigned to point at some actual function:

```
long (* pFunc1) (int, int);

long someFunc (int, int);//implemented somewhere
pFunc1 = someFunc; //pointers assignment
pFunc1(5,7);
```

** 11.1 pointer to functions
** 11.2 pointer to functions

Copyright @ 2018 Real Time College

Structures

Copyright @ 2018 Real Time College

Structures

Structure is a method of packing data of different types, a convenient method of handling a group of related data items of different data types.
A structure type is defined by

```
struct struct-name
{
    type field-name;
    type field-name; ...
}
```

Example:

```
struct lib_books
{
    char title[20];
    char author[15];
    int pages;
    float price;
};
```

Another example:

```
struct student_type
{
    char name[20];
    int ID;
}
```


Copyright @ 2018 Real Time College

Structures

the keyword struct declares a structure to holds the details of four fields namely title, author pages and price.

Each member may belong to different or same data type.

The structure we just declared is not a variable by itself but a template for the structure (a type)

We can declare variables of this structure by
struct lib_books book1,book2,book3;

declares book1,book2,book3 as variables of type struct lib_books each declaration has four elements of the structure lib_books..

You can make the the structure into a type:

typedef struct lib_books { …. } lib_books_t

Copyright @ 2018 Real Time College

Accessing Structure members Directly

to declare a variable of the Structure and access a member, use the operator '.' Which is known as dot operator or period operator.

```
lib_books book1;

book1. price = 15;
strcpy(book1.title,"basic");
strcpy(book1.author,"Balagurusamy");
book1.pages=250;
```

****Lesson-12.1**

Copyright @ 2018 Real Time College

Accessing Structure members Indirectly

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

To declare a pointer variable of the Structure and access a member within the class, use the operator '->'.

```
lib_books  book1,*ptr_book1 = &book1;

ptr_book1-> price = 15;
strcpy(ptr_book1-> title,"basic");
strcpy(ptr_book1-> author,"Balagurusamy");
ptr_book1-> pages=250;
```

****Lesson-12.2**

****Create an Employee managing program**

Copyright @ 2018 Real Time College

Init Structures

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

To declare a pointer variable of the Structure and access a member within the class, use the operator '->'.

```
lib_books  book1, book2 = {30, "IT", "Stiven King", 300};

book1.price = 15;
strcpy(book1.title,"basic");
strcpy(book1.author,"Balagurusamy");
book1.pages=250;
```

****Lesson-12.2**

****Create an Employee managing program**

144

Copyright @ 2018 Real Time College

Accessing Data Members

For an instance of an object - You accessed data members and functions by using the dot (`.`) operator

For a ptr to an instance of an object (access the object on the free store), C/C++ provides a shorthand operator for indirect access:

the `points-to` operator (`->`),

pCat->GetAge();

You can also use

*(*pCat).GetAge();*

Parentheses are used to assure that `pCat` is dereferenced before `GetAge()` is accessed.

**8.6 accessing member variables on the free store.

Copyright @ 2018 Real Time College

Pragma directive

The Compiler generates 32-bit code, and in that mode, there is a significant penalty (in terms of run-time performance) for unaligned accesses, like accessing a 16-bit short which isn't aligned on a 16-bit word boundary, or accessing a 32-bit int which isn't aligned on a 32-bit dword boundary.

To produce faster code, The Compiler pads struct members so that each one can be accessed without delays; this sometimes produces struct size which is larger than the sum of the sizes of its members. If you need to configure the exact struct with no padding you can use one of these directives used for that purpose:

Visual Studio: #pragma pack(1);
GCC Linux: use the __attribute__((packed)) extension

This will prevent The Compiler from padding the structure members; this will make accesses to some of the members significantly slower.

****Lesson-12.3**

Copyright @ 2018 Real Time College

Bitwise Operations

Copyright @ 2018 Real Time College

Bitwise Operations

- Usage:
  - data compression & encoding
  - Optimize computations
  - Many programs (***e.g.*** systems type applications) must actually operate at a low level where individual bits/bytes must be operated on.
  - The combination of pointers and bit-level operators makes C useful for many low level applications and can almost replace assembly code. (Only about 10 % of UNIX is assembly code the rest is C!!!)

Copyright @ 2018 Real Time College

Bitwise Operations

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

OR

X|Y	Y	X
0	0	0
1	1	0
1	0	1
1	1	1

AND

X&Y	Y	X
0	0	0
0	1	0
0	0	1
1	1	1

XOR

X^Y	Y	X
0	0	0
1	1	0
1	0	1
0	1	1

Table: Bitwise operators

Bitwise AND	&
Bitwise OR	|
Bitwise XOR	^
Bitwise NOT	~
Shift left	<<
Shift right	>>

Copyright @ 2018 Real Time College

Bitwise Shift Operators

The shift operators shift their first operand left (<<) or right (>>) by the number of positions the second operand specifies.

```
x = 0x1;
y = x << 1;  // y == 0x2
y = x >> 1;  // y == 0x1
```

You can also emulate multiplication and division by powers of two:

To multiply by 2 shift left once

To Divide by 2 shift right once

(Note: Avoid replacing multiplication and division with shifts, in general you should keep your code readable, and let the compiler optimize. Use shifts when you intend to do binary shifts)

Example 1:

```
//shift left
int main()
{
    int i = 1;
    int j = i << 5;
    printf("%d\n", j);
}
```

****Lesson-13.1**

****Lesson-13.2**

Copyright @ 2018 Real Time College

Changing Specific Bits

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

//set Register's bit 5 to Down state
//set Register's bit 3 to Up state

Example 2:

```
#define BitDown  5
#define BitUp 3

int main(void)
{
    //in order to access Register we must declare a pointer to it
    int* preg = (int*)REG_ADDR;

    //set to zero the bit 5
    *preg &= ~(1<<BitDown);

    //set to one  bit 3
    *preg |= (1<<BitUp);

    return 0;
}
```

**Lesson-13.1
**Lesson-13.2

Copyright @ 2018 Real Time College

Read Modify Write

Accessing and setting specific bits is often termed as a read modify write access, since you don't want to change the whole byte but just specific bits

For instance:

- (see the last page) setting bit 5 down and bit 3 up in some register, stored at address REG_ADDR.

- Suppose we want to assign bits[12:6] from some 16bit source register, to bits [6:0] of another 16bit target register.

	[15:12]	[11:8]	[7:4]	[3:0]
Source	110*1*	*1100*	*10*11	1010
Target	1010	1011	1*100*	*1101*
Result	1010	1011	1*111*	*0010*

Copyright @ 2018 Real Time College

Big & Little Endians

Copyright @ 2018 Real Time College

Big Endian

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Embedded Architectures differ in the way they store data in memory.

Two methods are used **Big Endian** and **Little Endian**

Examples of storing the value 0x0A0B0C0D in memory:

Addr	Value
0xF000	0x0A
0xF001	0x0B
0xF002	0x0C
0xF003	0x0D
0xF004	

In Big-Endian the most significant byte (MSB) value, which is 0x0A in our example, is stored at the memory location with the lowest address, the next byte value in significance, 0x0B, is stored at the following memory location and so on. This is akin to Left-to-Right reading in hexadecimal order.

Architectures such as Motorola, MIPS use this method

Copyright @ 2018 Real Time College

Little Endian

storing the same value 0x0A0B0C0D in memory:

Littel-endian

The least significant byte (LSB) value, 0x0D, is at the lowest address. The other bytes follow in increasing order of significance.

Architectures such as Intel use this method

Addr	Value
0xF000	0x0D
0xF001	0x0C
0xF002	0x0B
0xF003	0x0A
0xF004	

Question: write a function that returns true if the Architecture is Big-Endian else false if its Little Endian.

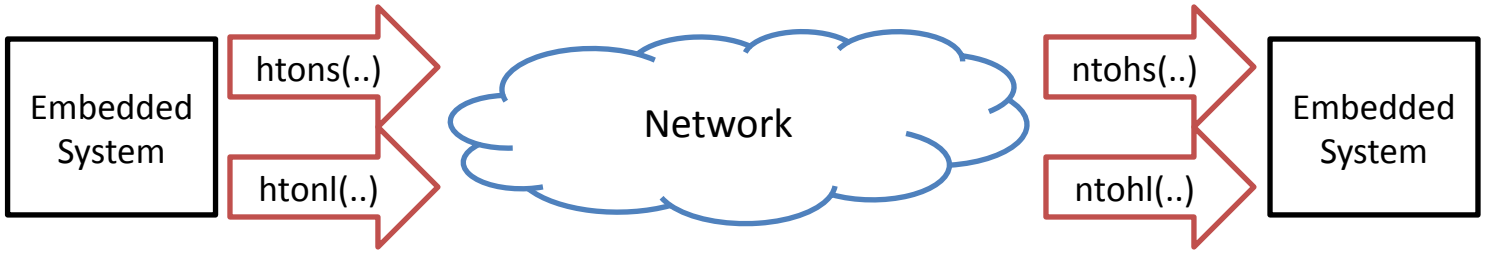
Copyright @ 2018 Real Time College

Endianness in networking

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Networks use Big-Endian order, and thus it is called network order.
when sending information over a network in a common format. the Internet Protocol defines a standard big-endian network byte order. This byte order is used for all numeric values in the packet headers and by many higher level protocols and file formats that are designed for use over IP.

The Berkeley sockets API defines a set of functions to convert 16- and 32-bit integers to and from network byte order: the *htonl* (host-to-network-long) and *htons* (host-to-network-short) functions convert 32-bit and 16-bit values respectively from machine (host) to network order; whereas the *ntohl* and *ntohs* functions convert from network to host order.



Copyright @ 2018 Real Time College

Bit Fields & Unions

Copyright @ 2018 Real Time College

Bit-Fields

We use bit fields as an effective alternative approach for storing and modifying data on a bit resolution instead of using masks & bitwise operators.

```
typedef struct Bit {
        char f1:1 ;
}Bit;

typedef struct Byte
{
        Uchar f1:1 ;//LSB
        Uchar f2:1 ;
        Uchar f3:1 ;
        Uchar f4:1 ;
        Uchar f5:1 ;
        Uchar f6:1 ;
        Uchar f7:1 ;
        Uchar f8:1 ;//MSB //sizeof()= 1 byte
}Byte;
```

**bitFields_examp.cpp

Copyright @ 2018 Real Time College

BitFields –cont'd

- Access members as in usual structures:

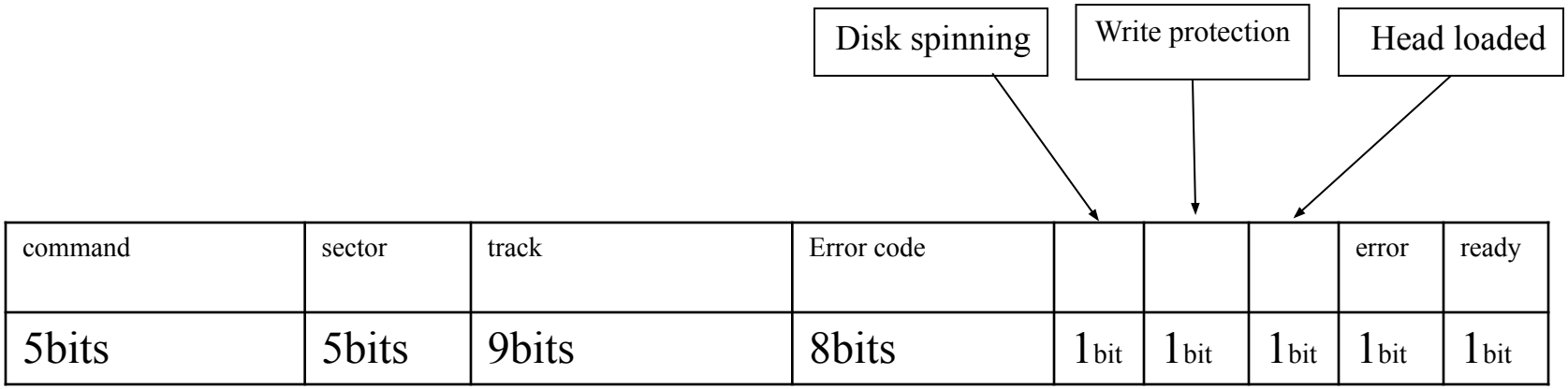
```
Byte MyByte.f2 =1;
Byte* pMybyte = &MyByte;
pMybyte->f2 = 0;
```

- type of bit field must be large enough for number of bits in that field.
- The first declared fields are the LSBs (in case it's little endian).
- Bit fields are <u>always</u> converted to integer type for computation.
- The unsigned definition is important - ensures that no bits are used as a + - flag.

Copyright @ 2018 Real Time College

BitFields –cont'd

A more "embedded" motivation:
Frequently device controllers (*e.g.* disk drives) and the operating system need to communicate at a low level. Device controllers contain several *registers* which may be packed together in one integer.



Copyright @ 2018 Real Time College

BitFields –cont'd


```
typedef struct DISK_REGISTER
{
    unsigned ready:1;
    unsigned error_occured:1;
    unsigned disk_spinning:1;
    unsigned write_protect:1;
    unsigned head_loaded:1;
    unsigned error_code:8;
    unsigned track:9;
    unsigned sector:5;
    unsigned command:5;

} DISK_REGISTER ;
```


Copyright @ 2018 Real Time College

BitFields –cont'd

To access values stored at a particular memory address, DISK_REGISTER_MEMORY, we can assign a pointer of the above structure to access the memory via:

```
volatile DISK_REGISTER* disk_reg = (volatile DISK_REGISTER *)DISK_REGISTER_MEMORY;
```

- The disk driver code to access this is now relatively straightforward:

```
/* Define sector and track to start read */
disk_reg->sector = new_sector;
disk_reg->track = new_track;
disk_reg->command = READ;

/* wait until operation done, ready will be true */
while ( ! disk_reg->ready ) ;

/* check for errors */
if (disk_reg->error)
{
     /* interrogate disk_reg->error_code for error type */
     switch (disk_reg->error_code)
     ......
}
```


Copyright @ 2018 Real Time College

BitFields – Notes of Caution

Sizeof(): there's no way to get the size of a certain bit field, **sizeof**(**MyByte.f2**) neither at run-time nor at compile time.

•Portability**:**

–**sizeof(struct x)** : in general, sizeof(struct) may be larger than the expected memory sum of its members because compilers tend to pad spaces in order to align all members to the optimal alignment CPUs best work with. (Optimal – in terms of speed and storage management efforts invested on the data type, whenever it's accessed to read/write).

–In order to force the compiler to pack data the way we want, there's a set of complier's directives: #pragma pack(n), __unaligned and more… note however that misaligned variables not in their natural boundaries may result in slower performances, and an effort of the OS to handle fix ups.

Copyright @ 2018 Real Time College

BitFields -Notes of Caution-cont'd

- **sizeof(struct BitStrct):** the amount of space such bit fields occupy and the policy used to align them in memory is very compiler (and sometimes OS) dependent. Some compliers pack all fields as compact as they can (when not all of the space of type of bit field was used), some do not.

- Big/little endian: Some bit field members are stored left to right others are stored right to left in memory.

- If portability of code is a premium, you can use bit shifting and masking to achieve the same results but not as easy to express or read.

- more details about alignment policies:
  http://publib.boulder.ibm.com/infocenter/lnxpcomp/v8v101/index.jsp?topic=/com.ibm.xlcpp8l.doc/proguide/ref/calgnbit.htm

Copyright @ 2018 Real Time College

Unions

- A union, is a collection of variables of different types, just like a structure. However, with unions, you can only store information in one field at any one time.
- The size of a union is equal to the size of it's largest data member
- Assigning a value to its biggest member will consequently overwrite all the values of the rest of the members.

Example 1:

```
//declaration+ instantiation of some union "at one shot"
union time{
        long simpleDate; //[sec]
        double perciseDate; //[msec]
} mytime;
```

Accessing is the same as in structures:

```
. . .
printTime( mytime.perciseDate );
. . .
```

**union_examp1.cpp

Copyright @ 2018 Real Time College

Unions-cont'd

A more "low level" motivation:

c[0] [7:0] **c[1] [15:7]** **c[2] [23:16]** **c[3] [31:24]**

lo[15:0] **hi[31:16]**

Long[31:0]

mix

```
union mix_t {
    long l;
    struct   {
        short lo;
        short hi;
    } s;
    char c[4];
} mix;
```

****union_examp2.cpp**

Copyright @ 2018 Real Time College

Unions-Notes of caution:

- *In the above example, we must be sure that our compiler allocates the same size to the long and the struct's members, so that they really share the entire same memory space.*

- *Big/little endian - We also need to confirm the order of MSBs/LSBs of the fields inside the union.*

- *The exact alignment and order of the members of a union in memory is platform dependant. Therefore be aware of possible portability issues with this type of use.*

Copyright @ 2018 Real Time College

IO commands

Copyright @ 2018 Real Time College

Basic IO commands (1)

Background

- Input device: Any object which produces a stream of bits, that can be read in into the program serially: keyboard, files (on Disk) , NIC, etc..
- Output device: any object which knows how to receive bits stream, into which this bits can be fed serially :screen, files (on Disk), NIC
- A C application can "speak" with several IO devices.

streams

Streams are a portable way of reading and writing data. They provide a flexible and efficient means of I/O.

A Stream is a **file or a physical device** (***e.g.*** printer or monitor) which is manipulated with a **pointer** to the stream.

Copyright @ 2018 Real Time College

FILE structure

FILE is the main file control structure for streams. The exact structure of it is very platform-dependent, so ANSI C proposes that the exact structure of this structured type should not be known, and well-written programs do not need to access the internal fields of this structure.

```
typedef struct {
    char *fpos;                 /* Current position of file pointer (absolute address) */
    void *base;                 /* Pointer to the base of the file */
    unsigned short handle; /* File handle */
    short flags;                /* Flags (see FileFlags) */
    short unget;                /* 1-byte buffer for ungetc (b15=1 if non-empty) */
    unsigned long alloc;        /* Number of currently allocated bytes for the file */
    unsigned short buffincrement; /* Number of bytes allocated at once */
} FILE;

enum FileFlags { _F_READ = 0x0001, _F_WRIT = 0x0002, _F_RDWR = 0x0003,
_F_ERR = 0x0010, _F_EOF = 0x0020, _F_BIN = 0x0040 };
```


Copyright @ 2018 Real Time College

FILE API's

Opening and Closing Files:

*FILE *fopen(const char *path, const char *mode);*
*int fclose(FILE * stream);*

Basic Reading and Writing:

*size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);*
*size_t fwrite(const void *ptr, size_t size, size_t nmemb,FILE *stream);*

Status Functions:

*void clearerr(FILE *stream);*
*int feof(FILE *stream);*
*int ferror(FILE *stream);*

Copyright @ 2018 Real Time College

Basic IO commands (2)

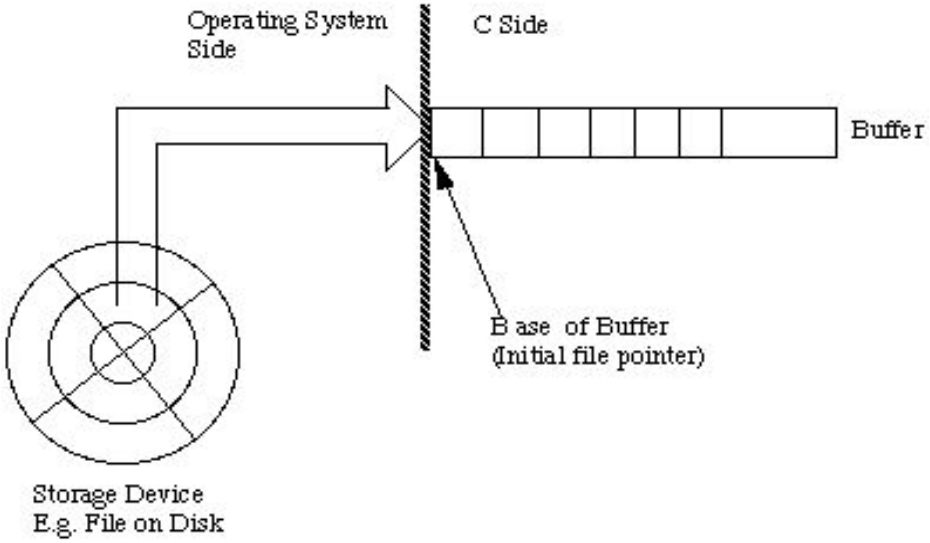
- There exists an internal C data structure, FILE, defined in stdio.h which identifies a stream and contains the information needed to control it, including
  - a pointer to its buffer.
  - its position indicator
  - its state indicators.
- We simply need to refer to the FILE structure in C programs when performing I/O with streams.
- We just need to declare a variable or pointer of this type in our programs.
- We do not need to know (at this stage) any more specifics about this definition. The program doesn't even know at runtime where its standard devices are linked to, it's the OS responsibility.
- We must open a stream before doing any I/O, then access it and then close it.
- Stream I/O is **BUFFERED**: That is to say a fixed "chunk" is read from or written to a file via some temporary storage area (the buffer).

Copyright @ 2018 Real Time College

Basic IO commands (3)

***Note*:**

- data written to a buffer does not appear in a file (or device) until the buffer is flushed or written out. (\n does this or endl in C). Any abnormal exit of code can cause problems.
- A data can be flushed "manually" using fflush(FILE*) command.



Copyright @ 2018 Real Time College

Standard streams

UNIX defines 3 predefined streams (in stdio.h):
stdin, stdout, stderr -They all use text as the method of I/O.

- *stdin* and *stdout* can be used with files, programs, I/O devices such as keyboard, console, ***etc.***.
- *stderr* always goes to the console or screen.
- The console is the default for *stdout* and *stderr*.
- The keyboard is the default for stdin.

Predefined streams are automatically open. When we run the program, we can temporarily change the IO devices with which it communicates by redirection:

```
hello < infile.txt  > outfile.txt
prog1 | prog2
out | lpr
```

The basic lib that deals with IO streams is stdio.h in C or iostream in C++.

Copyright @ 2018 Real Time College

Common functions for basic IO facilities

- `int getchar(void)` -- reads a char from stdin
- `int putchar(char ch)` -- writes a char to stdout, returns character written.

Related Functions:

```
int getc(FILE* stream);
int putc(char ch, FILE* stream);
```

Notes:

- At the first time the program needs input, OS stops the run and prompts user with blinking cursor.
- User types input and presses ENTER. Only after it presses ENTER, all the typed data is written to the defined stream, and program continues running.
- Now the program reads from the buffer as much as data as it needs. If user typed more than needed at the last read request, the rest of the data remains in the buffer for future stdin calls.
- EOF - End Of File –defined at stdio.h: *#define EOF -1*
- Ctrl +Z followed by a newline → artificial EOF (On windows. On Linux use Ctrl + D)

Copyright @ 2018 Real Time College

scanf() : a wise input command


```
int scanf(char* format, args....)
```

Reads data from stdin and stores them according to the parameter format into the locations pointed by the additional arguments.
The additional arguments should point to already allocated objects of the type specified by their corresponding format tag within the format string.

Example:

```
scanf("%7d %s %c %lf",&a, str, &ch, &x);
```

The above format string scans the first seven characters as a decimal integer, then reads the remaining as a string until a space, new line or tab is found, then scans the first non-whitespace character following and a double-precision floating-point number afterwards.

More at: http://www.cplusplus.com/reference/clibrary/cstdio/scanf/

** sscanf_examp.cpp

Copyright @ 2018 Real Time College

scanf() : Related Functions

- int sscanf (const char * str, const char * format, ...);
  similar to scanf() except that data is read from a string: str.

- int fscanf (FILE* stream, const char* format, ...);
  Similar to scanf() but reads formatted data from a stream not necessarily stdin.

Copyright @ 2018 Real Time College

printf() : a wise output command

int printf (const char format, ...);*
Writes to the standard output (stdout) a sequence of data formatted as the *format* argument specifies.
After the *format* parameter, the function expects at least as many additional arguments as specified in *format*.

Example:

```
printf(
    "Color %s, number1 %d, number2 %05d, "
    "hex %x, float  %5.2f, unsigned value %u.\n",

    "red",  123456, 89,
     255,   3.14159, 250
);
```

//output:

```
Color red, number1 123456, number2 00089, hex ff, float
3.14, unsigned value 250.
```


Copyright @ 2018 Real Time College

Printf() - Related Functions:

- *int sprintf (char* str, const char* format, ...);*
  similar to printf() except that data is written to **a string:** str**.**

- *int fprintf (FILE* stream, const char* format, ...);*
  Write formatted output to a stream not necessarily stdout.

** sprintf_examp.cpp

Copyright @ 2018 Real Time College

Getting and Putting an entire line at "one shot"

- *char* gets (char* str);*
  Reads characters from *stdin* and stores them as a string into *str* until a newline character ('\n') or the End-of-File is reached.

- *int puts (const char* str);*
  Writes the C string pointed by *str* to *stdout* and appends a newline character ('\n').

Caution note:

- gets( ) might write to str more than it is allowed: must be careful with the size of the array pointed by *str* to avoid buffer overflows!

- Low level IO – writing raw data bytes: each read/write request results in accessing disk (or device) directly to fetch/put a specific number of bytes. Data in this form is not buffered

** gets_examp.cpp

Copyright @ 2018 Real Time College

the extern Keyword

Copyright @ 2018 Real Time College

Extern keyword (1)

External variables are variables at file scope. They are defined outside the function and used inside it.

extern declares a reference to a variable defined elsewhere**.**
Once you have declared a reference to the variable at the external level, the variable is visible throughout the remainder of the translation unit in which the declared reference occurs.

For an **extern** reference to be valid, the variable it refers to must be defined once, and only once, at the external level. This definition can be in any of the translation units that make up the program.

Copyright @ 2018 Real Time College

Extern keyword (2)

The extern keyword declares a variable or function and specifies that it has external linkage (its name is visible from files other than the one in which it's defined).

When modifying a variable, extern specifies that the variable has static duration (it is allocated when the program begins and de-allocated when the program ends).

The variable or function may be defined in another source file, or later in the same file. Declarations of variables and functions at file scope are external by default.

a function prototype using **extern** is only optional (i.e. default), since functions can only be defined at the external level and cannot be nested. This means they have "external linkage" by default. (Functions from other files are "known" to the compiler when including their header files, signaling the compiler they would

be linked later on).

Copyright @ 2018 Real Time College

Extern keyword (3)

Behind the scenes :

- a **symbol table** is a data structure used by the **compiler**, holding information about each identifier in the program's source code . That info can be its type, scope level and sometimes its location**.**

- An **object file** will contain a symbol table of the identifiers it contains that are externally visible. During the linking of different object files, a **linker** will use these symbol tables to resolve any unresolved references.

- **extern** tells the compiler that the variable will be found in the symbol table of a different translation unit.

- The external declaration is not resolved by the **compiler**, but instead by the **linker**: modularity at the compiler level.

Copyright @ 2018 Real Time College

Static Variables

Copyright @ 2018 Real Time College

Static Variables (1)

A **static variable** is a variable that has been allocated statically and whose lifetime extends across the entire run of the program.

Generally, static is a special type of local variable, called a static local, which:

- is local to particular function (Outside the function can't be accessed.)
- is only initialized once (on the first call to function).
- On the next call to the function, the static variable has the same value as on leaving.

* In this case of recursive calls to the function, it also has access to the single, statically allocated variable.

*If the function is global, the **static** variable declared in it is as a global variable in terms of space scope, except that its scope is only within that function.

Copyright @ 2018 Real Time College

Static Variables (2)

Example 1:

```
int running_total()
{
  static int sum = 0;
  sum ++;
  return sum;
}
```


Copyright @ 2018 Real Time College

Static Variables (3)

Example 2:

```
void foo(void)
{   static int first=1;  /* first is true */

if (first)
        {       ... /* do something */
                first = 0;  /* now set it to false */
        }
        else
        {
                ... /* do something else */
        }
 }
```


Copyright @ 2018 Real Time College

Static Variables (4)

Notes:

- Automatic variables are initialized during the run time. Static variables are initialized during the compile-time, so the initial values will simply be embedded in the executable file itself.
- If you change them, they will retain changed in the file.

Copyright @ 2018 Real Time College

Conclusion:

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

space scope	time scope/storage class	variable type
block	automatic	**local**
block	static	**static local**
current translation object	static	**global**
current or other translation objects of the program	static	**Extern**
depends on use	dynamic allocation	**malloc()/realloc()**

Copyright @ 2018 Real Time College

More Variables

Storage duration specifiers **:**

- **auto** – by default for any local variable, so rarely used**…**

- **register -** hint to the compiler to cache the variable in a processor register. Other than not allowing the referencing operator (&) to be used on the variable or any of its subcomponents, the compiler is free to ignore the hint. (Microsoft compiler ignores this accept, and uses it just for debugging & requests)

- **volatile -**used to declare that an object that might be modified outside the program by the operating system or the hardware or a concurrently executing thread.

  The system always reads (from RAM) the current value of a volatile object at the point it is requested, even if a previous instruction asked for a value from the same object. Also, the value of the object is written (to RAM) immediately on assignment - volatile signal to the compiler not to optimize use of the variable that is declared.

Copyright @ 2018 Real Time College

Linked Lists

Copyright @ 2018 Real Time College

Singly Linked list

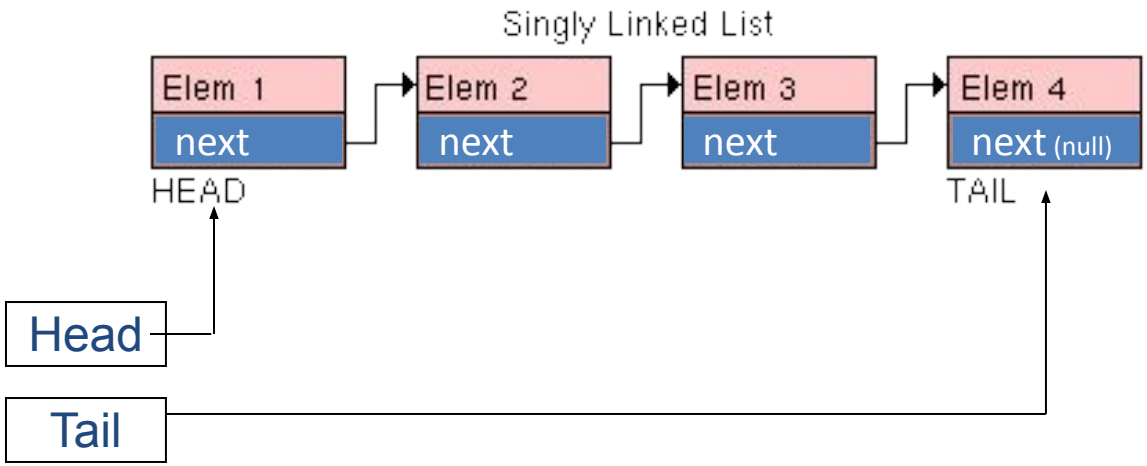
A linked list is a chain of structures which contain a pointer to the next element.
There are 3 main types of linked lists

1. Singly linked
2. Doubly linked
3. Trees

In a **singly linked list**, each node points forward to the next one, but not backward. Each Node includes:

- the Data (int /structure/class)
- the next Node

```
struct Node
{
        int elem;
        struct Node * next;
};
```



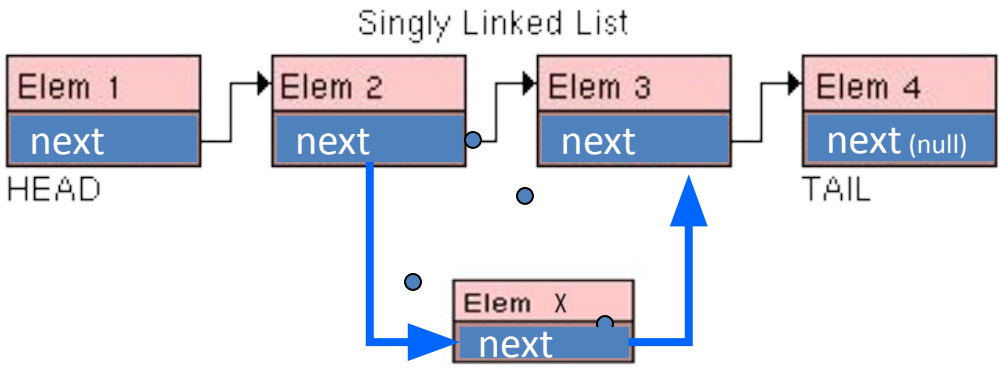
Copyright @ 2018 Real Time College

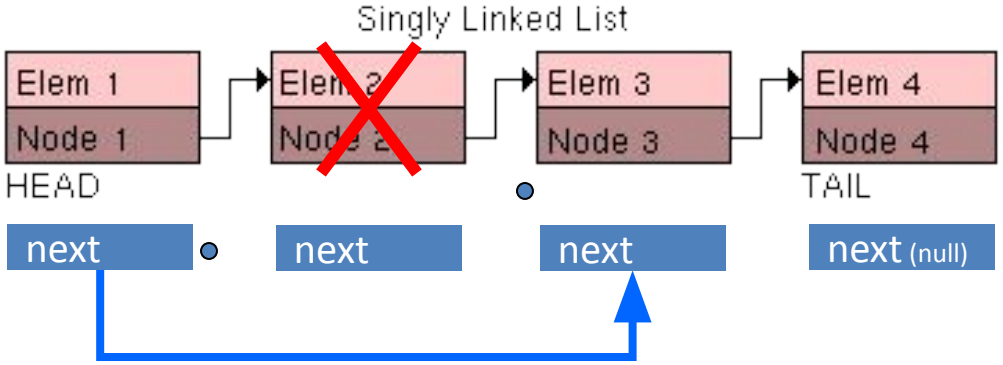
Singly Linked list

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

- Adding and removing an element in the middle of the linked list is simple and fast.

```
Add(Node *list, Elem new)
{
Node *newNode;
newNode = malloc (sizeof (*newNode));
newNode -> elem = new;
newNode -> next = list -> next;
list -> next = newNode;
}
Remove(Node *list)
{
        Node *temp = list -> next;
        list -> next = temp -> next;
        free (temp);
}
```





Copyright @ 2018 Real Time College

Linked list VS Array

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Array	Linked List	
O(1)	O(n) *	Indexing
Not Available	O(1)	Insert & Delete (in head / tail)
Not Available	SearchTime + Insert: O(n) + O(1)	Insert & Delete (in the middle)
0 Yet, fixed in size	(Node-Elem / Node) * N	Overhead

*** The amount of time to be indexed can be shorten by a hash table, which will be explained in depth on the following pages.**

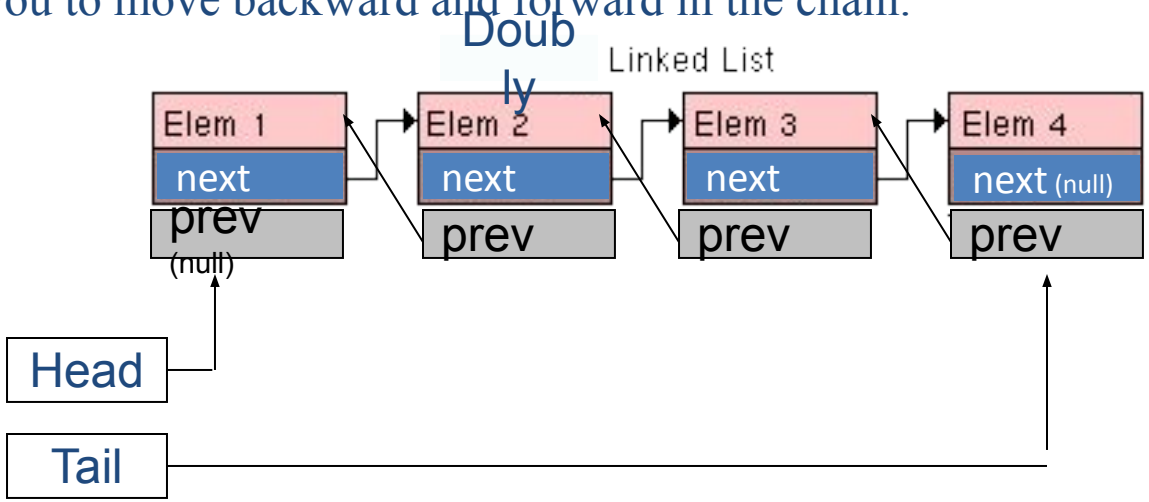
Copyright @ 2018 Real Time College

Doubly Linked list

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

A doubly linked list enables you to move backward and forward in the chain.

```
struct Node
{
        int Elem;
        struct Node * next1;
        struct Node  * prev;
};
```



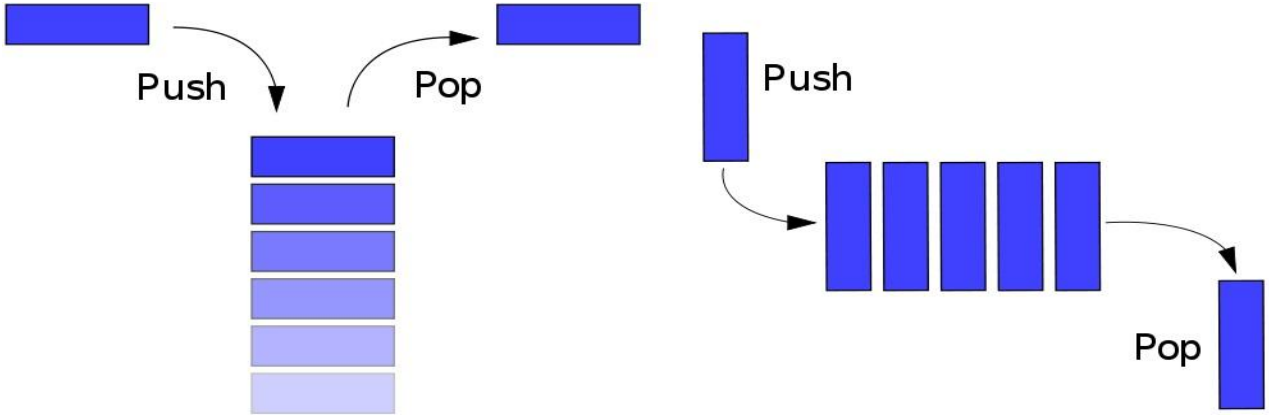
**queue

**linked_list

Copyright @ 2018 Real Time College

Using Stacks & Queues

- The abstract concept of stacks and queues does not require a specific data structure, there is no restriction of some sort.
- However, in most cases, it is recommended to implement both with linked list.
- Stacks and Queues must implement the two functions:
  - void push(element X)
  - element X pop()
- The terms used relating to stacks and queues are also FIFO and LIFO



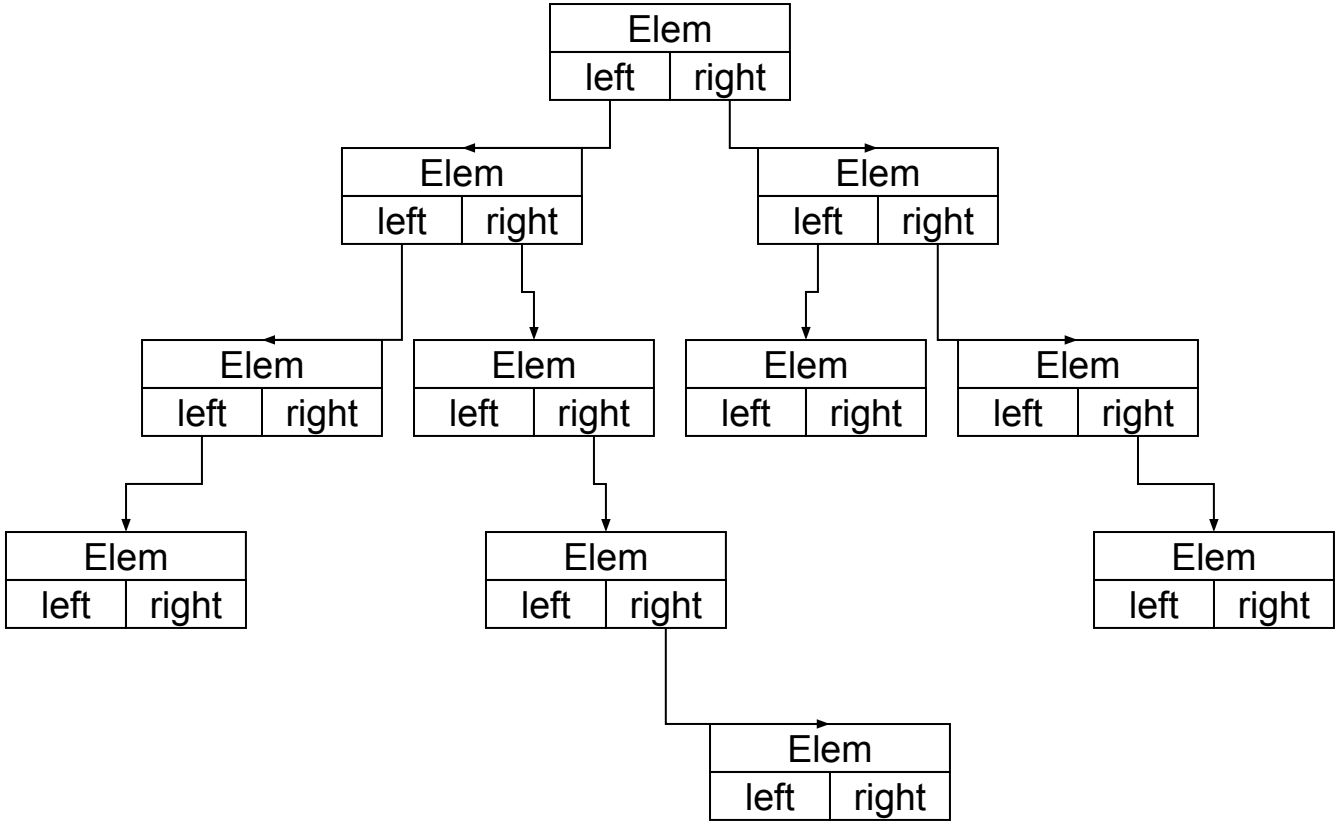
Copyright @ 2018 Real Time College

Binary Tree

A tree is a complex structure built from nodes, each of which can point in two or three directions

```
struct Node
{
int Elem;
struct Node * Left;
struct Node  *Right;
};
```

.



Copyright @ 2018 Real Time College

Hash Tables

Copyright @ 2018 Real Time College

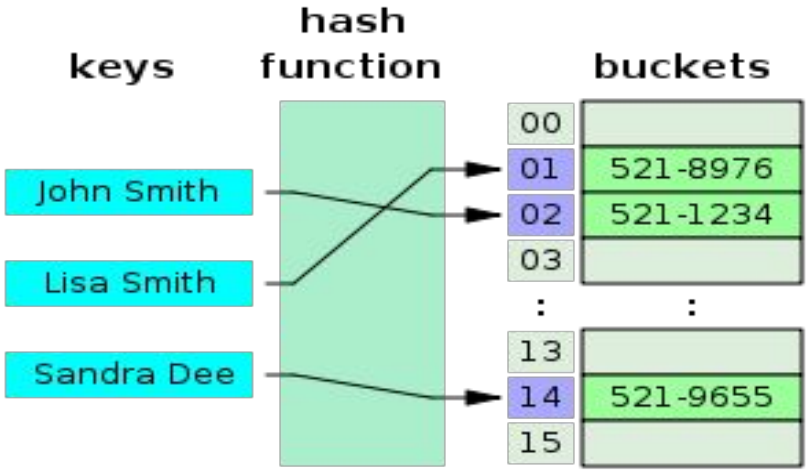
Hash Tables

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

The **hash table** or **hash map** is a data structure that uses a hash function to efficiently map certain identifiers or keys (e.g., person names) to associated values (e.g., their telephone numbers).

The **hash function** is used to transform the key into the index (the *hash*) of an array element (the *slot* or *bucket*) where the corresponding value is to be found.

Index = func (Key)



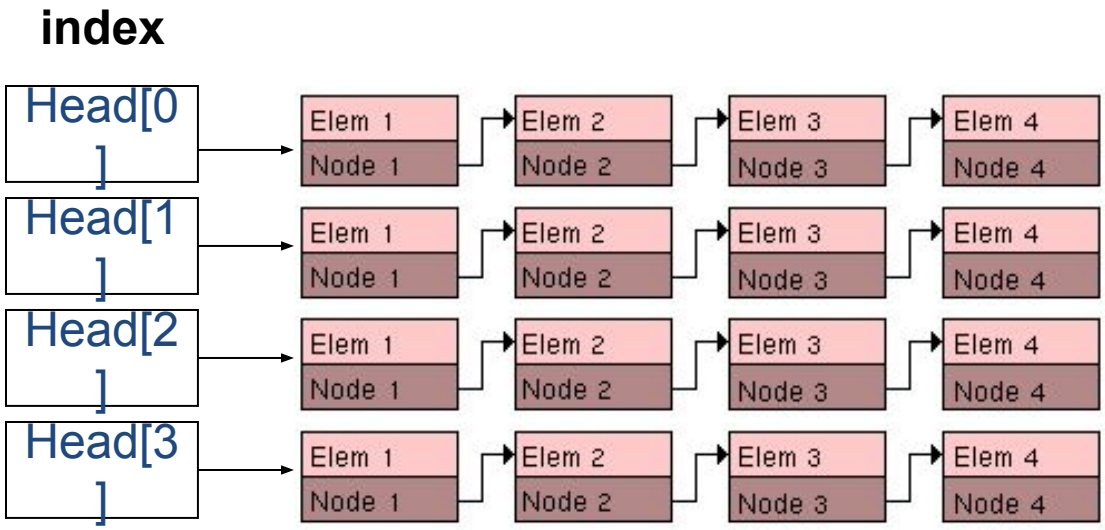
Copyright @ 2018 Real Time College

Hash Tables

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Ideally the hash function should map each possible key to a different slot index, but this is rarely achievable.

in practice Most hash table designs assume that *hash collisions* — pairs of different keys with the same hash values are normal occurrences and must be accommodated in some way

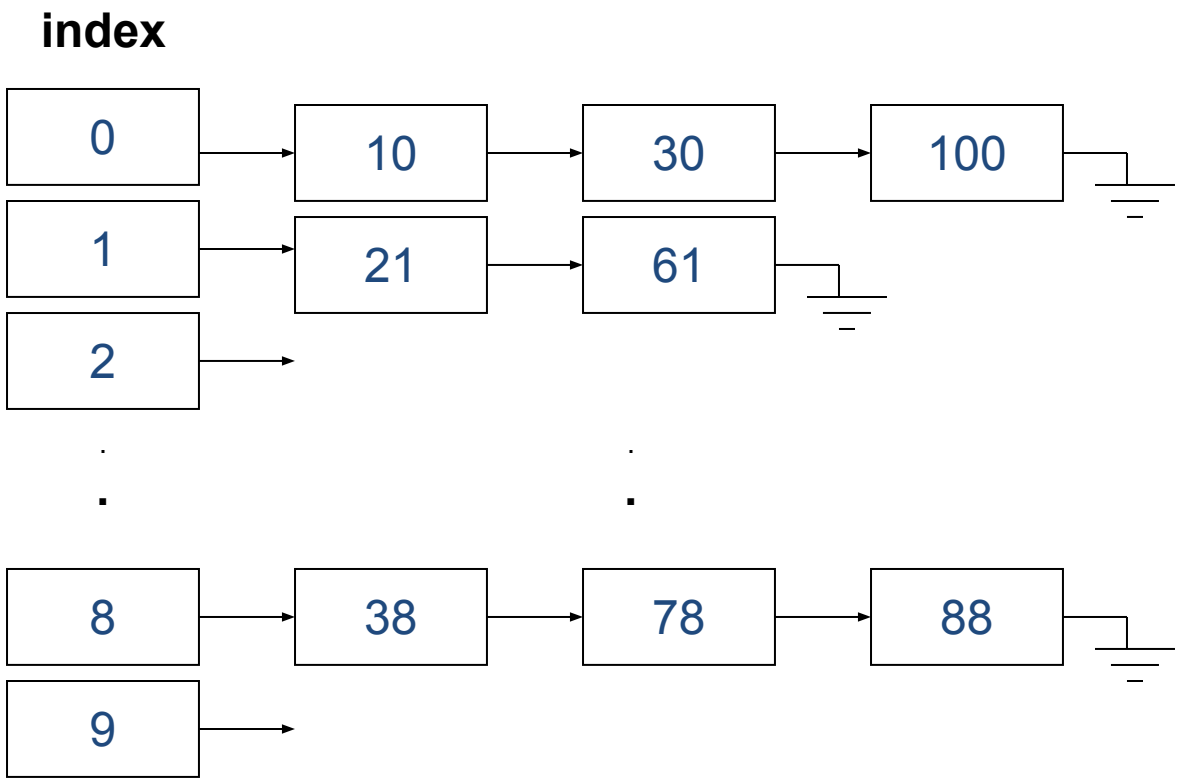


Copyright @ 2018 Real Time College

Hash Function example

We have an pool of numbers all between 1-100 which we want to be able to access in approximately O(1).

Index = Hash_Function (Number) => Index = Number % 10



Copyright @ 2018 Real Time College

Appendix A
Sorting Methods

Copyright @ 2018 Real Time College

Bubble sort

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

It is the simplest sorting algorithm. The idea is to pass through the list checking for the right order by comparing two adjacent items.
Each comparison results in smaller on the left, bigger on the right. Swap as needed.
After each iteration the biggest item will bubble up, thus each iteration of passing through the array will be smaller by 1, until the array is sorted.

Single iteration explanation:

(5 6 3 7 2 8 1 4)	5<6 ok
(5 6 3 7 2 8 1 4)	6>3 swap
(5 3 6 7 2 8 1 4)	6<7 ok
(5 3 6 7 2 8 1 4)	7>2 swap
(5 3 6 2 7 8 1 4)	7<8 ok
(5 3 6 2 7 8 1 4)	8>1 swap
(5 3 6 2 7 1 8 4)	8>4 swap
(5 3 6 2 7 1 4 8)	8 bubbled up

Processing N-1 iterations:

(5 6 3 7 2 8 1 4)	starting point
(5 3 6 2 7 1 4 8)	8 bubbled up
(3 5 2 6 1 4 7 8)	7 bubbled up
(3 2 5 1 4 6 7 8)	6 bubbled up
(2 3 1 4 5 6 7 8)	5 bubbled up
(2 1 3 4 5 6 7 8)	4 bubbled up
(1 2 3 4 5 6 7 8)	3 bubbled up
(1 2 3 4 5 6 7 8)	2 bubbled up

Copyright @ 2018 Real Time College

Insertion sort

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

It is a simple algorithm. Here, sorting takes place by inserting a particular item at the appropriate position.
In general, in every iteration an item is compared with all the items before it, when the suitable position is found the items whose bigger are shifted forward.
In each iteration one more item is inserted, the process is repeated for all items, until the list is sorted.

Single iteration explanation:

(5 6 3 7 2 8 1 4)	6 is in place
(5 6 3 7 2 8 1 4)	3 is processed
(5 3 6 7 2 8 1 4)	3 is processed
(3 5 6 7 2 8 1 4)	3 is in place
(5 3 6 7 2 8 1 4)	7 is in place

Processing N-1 iterations:

(5 6 3 7 2 8 1 4)	6 is inserted
(3 5 6 7 2 8 1 4)	3 is inserted
(3 5 6 7 2 8 1 4)	7 is inserted
(2 3 5 6 7 8 1 4)	2 is inserted
(2 3 5 6 7 8 1 4)	8 is inserted
(1 2 3 5 6 7 8 4)	1 is inserted
(1 2 3 4 5 6 7 8)	4 is inserted
(1 2 3 4 5 6 7 8)	sorted

Copyright @ 2018 Real Time College

Selection sort

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Yet, another simple algorithm, somewhat similar to Insertion sort.
The list is again devided into the sorted area and the unsorted area. Each iteration, the next lowest item is beeing selected, and swapped to the correct position.
In each iteration the selected item is added to the sorted list, the process is repeated for all items, until the list is sorted.

Single iteration explanation:

(5 6 3 7 2 8 1 4)	1 is selected
(1 6 3 7 2 8 5 4)	swapping [1 | 5]
(1 6 3 7 2 8 5 4)	2 is selected
(1 2 3 7 6 8 5 4)	swapping [2 | 6]
(1 2 3 7 6 8 5 4)	3 is selected
(1 2 3 7 6 8 5 4)	swapping [3 | 3]

Processing N-1 iterations:

(5 6 3 7 2 8 1 4)	1 is selected
(1 6 3 7 2 8 5 4)	2 is selected
(1 2 3 7 6 8 5 4)	3 is selected
(1 2 3 7 6 8 5 4)	4 is selected
(1 2 3 4 6 8 5 7)	5 is selected
(1 2 3 4 5 8 6 7)	6 is selected
(1 2 3 4 5 6 7 8)	7 is selected
(1 2 3 4 5 6 7 8)	sorted

Copyright @ 2018 Real Time College

Comparison

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Bubble Sort

Worst case performance	$O(n^2)$
Best case performance	$O(n)$
Average case performance	$O(n^2)$

Insertion Sort

Worst case performance	$O(n^2)$
Best case performance	$O(n)$
Average case performance	$O(n^2)$

Selection Sort

Worst case performance	$O(n^2)$
Best case performance	$O(n^2)$
Average case performance	$O(n^2)$

Copyright @ 2018 Real Time College

Shell sort

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

An expansion of Insertion and Bubble sort. Done by partially sorting items with a shrinking gap. The sorting can be very efficient on a large set of items.
Sorting is done several times, on sub-arrays of the main array. Each sub array is a group of items that lie far apart. Example – *items:[0,8,16,24] items:[1,9,17,25]* ..
The running time of Shellsort is heavily dependent on the gap sequence it uses.

Sorting explanation: Gap=4.

(5 6 3 7 2 8 1 4) 1st group
(5 6 3 7 2 8 1 4) 2nd group
(5 6 3 7 2 8 1 4) 3rd group
(5 6 3 7 2 8 1 4) 4th group

(2 6 1 4 5 8 3 7) partially sorted.

Sorting explanation: Gap=3.

(2 6 1 4 5 8 3 7) 1st group
(2 6 1 4 5 8 3 7) 2nd group
(2 6 1 4 5 8 3 7) 3rd group

(2 5 1 3 6 8 4 7) partially sorted.

Sorting explanation: Gap=1.

(2 5 1 3 6 8 4 7) regular sorting
(1 2 3 4 5 6 7 8) sorted

Copyright @ 2018 Real Time College

Shell sort performance

Worst case performance	depends on gap sequence
Best case performance	O(*N*)
Average case performance	depends on gap sequence
Worst case space complexity	O(1)

Copyright @ 2018 Real Time College

Merge sort

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

A sophisticated divide and conquer algorithm that uses two basic functions to operate. *merge-sort* to split, and *merge* to sort. Splitting is done recursively.
The splitting is done **recursively** with *merge-sort* until the size of the array is 1.
The merging is done **backwards**. It takes two already sorted sub-arrays and merges them

*First thing, understanding **merge**:*

- Lets assume we have two sorted sub-ararys:

(1 6 8 15 24 30) sub-array 1 (2 5 9 14 20 32) sub-array 2

- The next thing to do is to build a single sorted array which combines all items:

(1 6 8 15 24 30) sub-array 1 (2 5 9 14 20 32) sub-array 2
Item **1<2**, choosing 1 **(1)**
(1 6 8 15 24 30) sub-array 1 (2 5 9 14 20 32) sub-array 2
Item **6>2**, choosing 2 **(1 2)**
(1 6 8 15 24 30) sub-array 1 (2 5 9 14 20 32) sub-array 2
Item **6>5**, choosing 5 **(1 2 5)**
(1 6 8 15 24 30) sub-array 1 (2 5 9 14 20 32) sub-array 2

Copyright @ 2018 Real Time College

Merge sort (2)

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Understanding merge:

(1 6 8 15 24 30) sub-array 1 (2 5 9 14 20 32) sub-array 2

Item **6<9**, choosing 6 **(1 2 5 6)**

Item **8<9**, choosing 8 **(1 2 5 6 8)**

Item **15>9**, choosing 9 **(1 2 5 6 8 9)**

Item **15>14**, choosing 14 **(1 2 5 6 8 9 14)** **and so on …**

*Understanding **merge_sort**:*

Lets assume we have unsorted array. Lets define left, rigt and middle, in order to split.

Left, Middle, Middle+1, Right

23	13	6	72	24	15	40	38	9	10	62	99	31	56

Left, Middle, Middle+1, Right

23	13	6	72	24	15	40

Left, Right

38	9	10	62	99	31	56

Left, Right

23	13	6

Left, Right

72	24	15	40

Left, Right

23	13	6

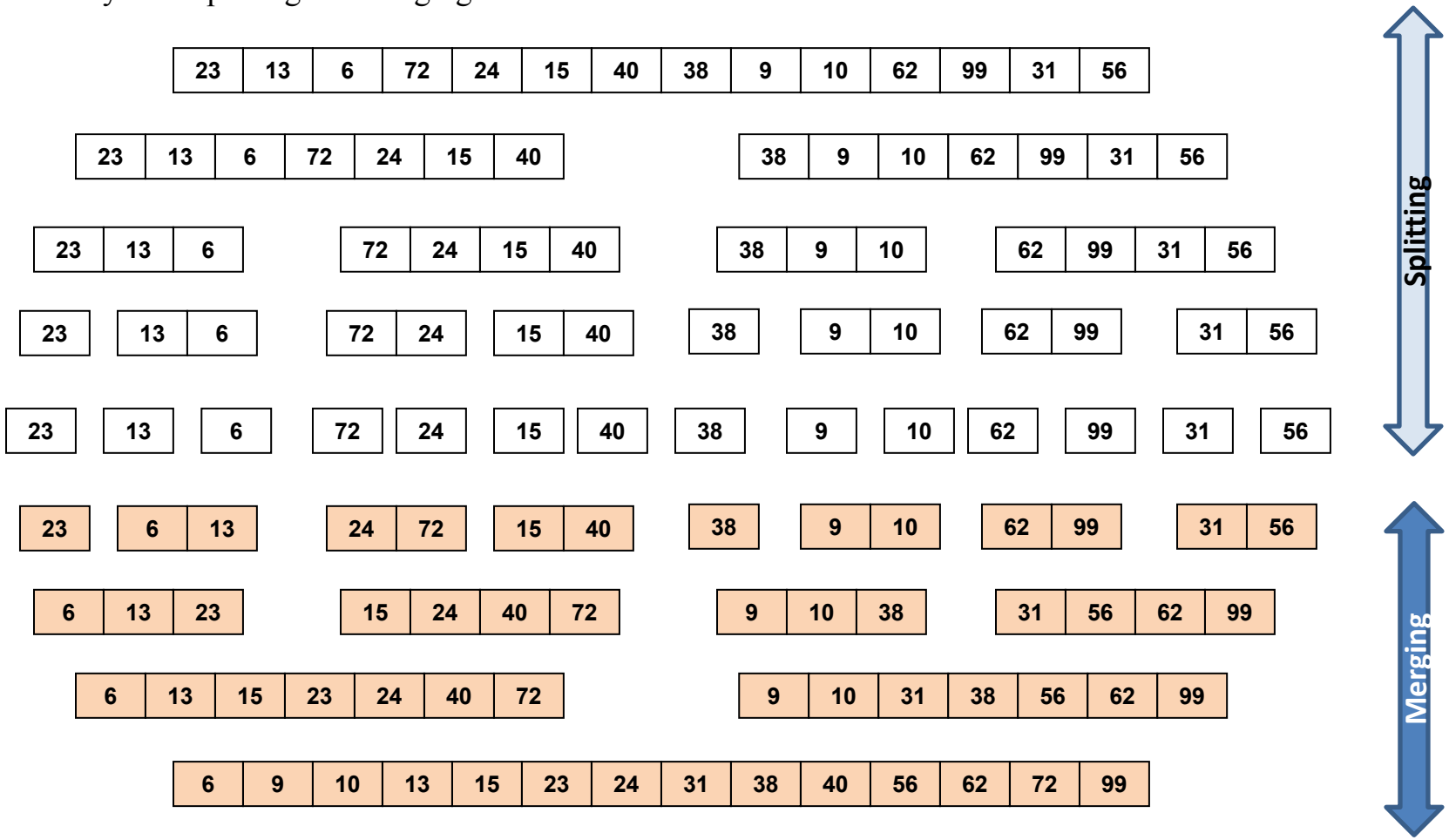
Copyright @ 2018 Real Time College

Merge sort (3)

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Understanding merge_sort:

Ultimately after splitting and merging the list will be sorted.



Copyright @ 2018 Real Time College

Merge sort performance

Worst case performance	O(n log n)
Best case performance	O(n log n) typical, O(n) natural variant
Average case performance	O(n log n)
Worst case space complexity	O(n) auxiliary

Parallel processing:

Merge sort can be multi processed well due to use of divide-and-conquer method.

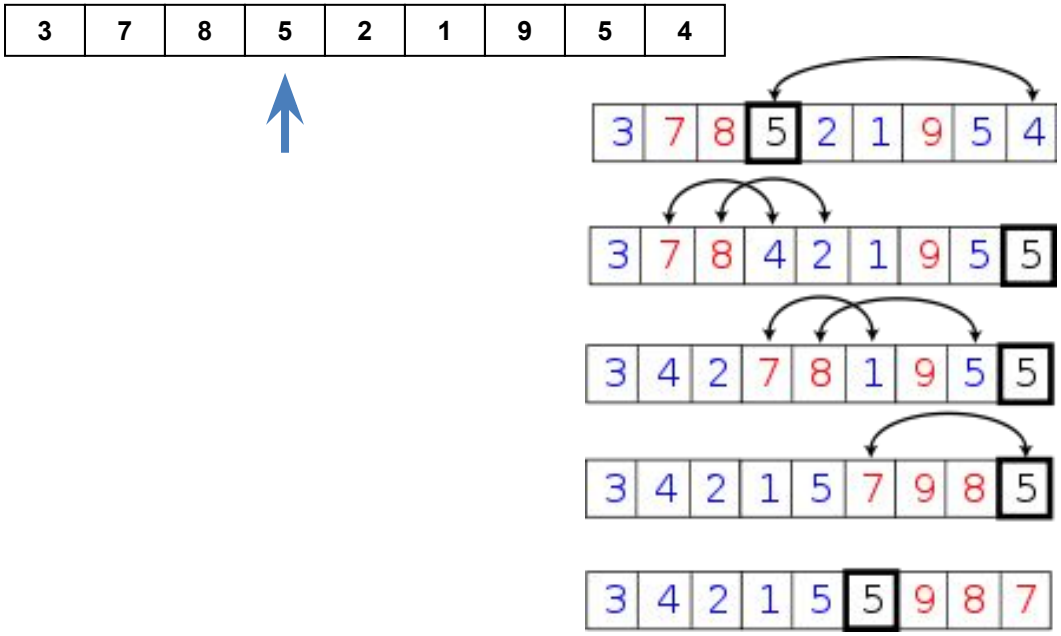
The multi processing can be done not only on the recursive division of the array, but also the merge operation.

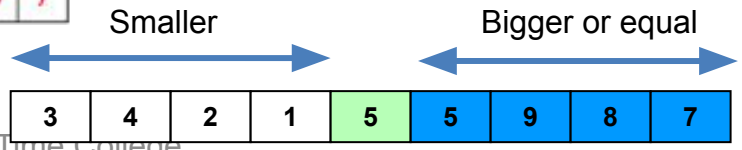
Copyright @ 2018 Real Time College

Quick sort

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Quicksort is a divide and conquer algorithm.
It first divides a large list into two smaller sub-lists: the low elements and the high elements. Then it continues the operation for each sub-list.
The transition of sorting the sub-lists is usually done **recursively**.





Copyright @ 2018 Real Time College

Quick sort

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Single iteration explanation:

(3 7 8 5 2 1 9 4) 5 is selected

working on 5 (5 is moved to the end)

(3 7 8 4 2 1 | 9 5) 7 > 5 and 1 <= 5

(3 1 8 4 2 | 7 9 5) swapped! next ..
8 > 5 and 2 <= 5

(3 1 2 4 | 8 7 9 5) time to end iter'

(3 1 2 4 | 5 7 9 8)

5 is moved to the start of the right half

Processing next iterations:

(3 1 2 4) (5 7 9 8) doing the same for each

Basically it's time to practice ..

(_ _ _ _) (_ _ _ _)
(_ _ _ _) (_ _ _ _)
(_ _ _ _) (_ _ _ _)
(_ _ _ _) (_ _ _ _)
(_ _ _ _) (_ _ _ _)
(_ _ _ _) (_ _ _ _)
(_ _ _ _) (_ _ _ _)

Copyright @ 2018 Real Time College

Quick sort performance

בית תוכנה, מרכז להכשרה מקצועית ומיקור חוץ

Worst case performance	$O(n^2)$
Best case performance	$O(n \log n)$
Average case performance	$O(n \log n)$
Worst case space complexity	$O(n)$

Parallelization:
Like merge sort, quicksort can also be parallelized due to its divide-and-conquer nature.

Individual in-place partition operations are difficult to parallelize, but once divided, different sections of the list can be sorted in parallel.

If we have *(p)* processors, we can divide a list of n elements into *p* sub-lists in *O(n)* average time, then sort each of these in $O\left(\frac{n}{p}\log\frac{n}{p}\right)$ **average time.**

Copyright @ 2018 Real Time College